



# legoESM v0.1: A Differentiable Earth System Model in JAX

Technical Documentation of Equations and Implementations

legoESM Development Team

Version 0.1 — June 2026

## Contents

|          |                                                                             |           |
|----------|-----------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                         | <b>6</b>  |
| <b>2</b> | <b>Model Architecture</b>                                                   | <b>6</b>  |
| <b>3</b> | <b>Grid Systems</b>                                                         | <b>7</b>  |
| 3.1      | Cubed-Sphere Grid . . . . .                                                 | 7         |
| 3.1.1    | Face-to-Cartesian mapping . . . . .                                         | 7         |
| 3.1.2    | Cell area . . . . .                                                         | 9         |
| 3.1.3    | Grid spacing . . . . .                                                      | 9         |
| 3.1.4    | Grid angle and wind rotation . . . . .                                      | 9         |
| 3.1.5    | Halo exchange . . . . .                                                     | 10        |
| 3.2      | Gaussian Grid and Spectral Transforms . . . . .                             | 10        |
| 3.2.1    | Grid dimensions . . . . .                                                   | 10        |
| 3.2.2    | Associated Legendre polynomials . . . . .                                   | 10        |
| 3.2.3    | Spherical harmonic transforms . . . . .                                     | 10        |
| 3.2.4    | Spectral operators . . . . .                                                | 11        |
| 3.2.5    | Velocity from vorticity and divergence . . . . .                            | 11        |
| 3.3      | Sigma Vertical Coordinate . . . . .                                         | 11        |
| 3.3.1    | Simmons–Burridge geopotential . . . . .                                     | 11        |
| 3.3.2    | Sigma-dot . . . . .                                                         | 11        |
| 3.3.3    | Pressure velocity . . . . .                                                 | 11        |
| 3.4      | Height and $z^*$ Coordinates . . . . .                                      | 12        |
| <b>4</b> | <b>Core Operators</b>                                                       | <b>12</b> |
| 4.1      | Gradient . . . . .                                                          | 12        |
| 4.2      | Divergence (flux form) . . . . .                                            | 12        |
| 4.3      | Vorticity . . . . .                                                         | 12        |
| 4.4      | C-D Grid Operators (FV3-style, <code>operators_cdgrid.py</code> ) . . . . . | 12        |
| 4.5      | Laplacian and Hyperdiffusion . . . . .                                      | 13        |
| <b>5</b> | <b>Finite-Volume Transport</b>                                              | <b>13</b> |
| 5.1      | MUSCL Reconstruction (2nd order, <code>halo = 1</code> ) . . . . .          | 14        |
| 5.2      | PPM Reconstruction (4th order, <code>halo = 2</code> ) . . . . .            | 14        |
| 5.2.1    | Fourth-order edge values . . . . .                                          | 14        |
| 5.2.2    | Monotonicity constraint . . . . .                                           | 14        |
| 5.2.3    | Colella–Woodward limiting . . . . .                                         | 14        |

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| 5.3      | WENO5 Reconstruction (5th order, halo = 2)               | 14        |
| 5.3.1    | Left-biased reconstruction at $i + \frac{1}{2}$          | 14        |
| 5.3.2    | Smoothness indicators                                    | 14        |
| 5.3.3    | Nonlinear weights                                        | 15        |
| 5.3.4    | Right-biased reconstruction                              | 15        |
| 5.4      | Rusanov Flux                                             | 15        |
| 5.5      | Well-balanced scheme                                     | 15        |
| <b>6</b> | <b>FC-Gram Spectral Operators</b>                        | <b>15</b> |
| 6.1      | Extension operator                                       | 15        |
| 6.2      | Spectral differentiation                                 | 15        |
| 6.3      | C-grid-style divergence damping                          | 16        |
| <b>7</b> | <b>Atmospheric Dynamics</b>                              | <b>16</b> |
| 7.1      | Shallow Water Equations                                  | 16        |
| 7.2      | Hydrostatic Primitive Equations ( $\sigma$ -coordinates) | 16        |
| 7.3      | Spectral Primitive Equations (Vorticity–Divergence)      | 17        |
| 7.4      | Non-Hydrostatic Compressible Euler Equations             | 17        |
| 7.4.1    | Split-explicit time integration                          | 17        |
| 7.4.2    | Sponge layer                                             | 18        |
| 7.4.3    | Turbulence closure: CRM, LES, and DNS                    | 18        |
| <b>8</b> | <b>Atmospheric Physics</b>                               | <b>18</b> |
| 8.1      | Gray Radiation (Frierson 2006)                           | 18        |
| 8.1.1    | Longwave optical depth                                   | 18        |
| 8.1.2    | Two-stream LW fluxes                                     | 19        |
| 8.1.3    | Shortwave (Beer–Lambert)                                 | 19        |
| 8.1.4    | Heating rate                                             | 19        |
| 8.2      | RRTMGP Correlated- $k$ Radiation                         | 19        |
| 8.3      | SBM Convection (Frierson 2007)                           | 19        |
| 8.3.1    | Moist adiabatic lapse rate                               | 19        |
| 8.3.2    | Saturation mixing ratio (Tetens)                         | 19        |
| 8.3.3    | CAPE and trigger                                         | 20        |
| 8.3.4    | Enthalpy-conserving relaxation                           | 20        |
| 8.3.5    | Precipitation                                            | 20        |
| 8.4      | Held–Suarez Forcing                                      | 20        |
| 8.4.1    | Equilibrium temperature                                  | 20        |
| 8.4.2    | Newtonian relaxation                                     | 20        |
| 8.4.3    | Rayleigh friction                                        | 20        |
| 8.5      | Louis (1979) Turbulence                                  | 20        |
| 8.5.1    | Mixing length                                            | 20        |
| 8.5.2    | Richardson number                                        | 20        |
| 8.5.3    | Stability functions                                      | 21        |
| 8.6      | Vertical Diffusion (Implicit Thomas Algorithm)           | 21        |
| 8.7      | Surface Fluxes                                           | 21        |
| 8.7.1    | Constant coefficients (default)                          | 21        |
| 8.7.2    | Monin–Obukhov Similarity Theory (MOST)                   | 21        |
| 8.7.3    | COARE 3.0 roughness (Fairall et al. 2003)                | 22        |
| 8.7.4    | Large & Yeager (2004) roughness                          | 22        |
| 8.8      | Additional Turbulence Schemes                            | 22        |
| 8.8.1    | Smagorinsky                                              | 22        |
| 8.8.2    | YSU (Yonsei University)                                  | 22        |

|           |                                                                        |           |
|-----------|------------------------------------------------------------------------|-----------|
| 8.8.3     | CLUBB-lite . . . . .                                                   | 22        |
| 8.8.4     | EDMF Turbulence . . . . .                                              | 22        |
| 8.9       | Additional Convection Schemes . . . . .                                | 23        |
| 8.9.1     | Kuo (1974) . . . . .                                                   | 23        |
| 8.9.2     | Mass-Flux Convection (Arakawa–Wu Type) . . . . .                       | 23        |
| 8.9.3     | EDMF Convection . . . . .                                              | 23        |
| 8.9.4     | Deep Convective Adjustment (DCA) . . . . .                             | 23        |
| 8.10      | Gravity Wave Drag . . . . .                                            | 23        |
| 8.11      | Cloud Microphysics . . . . .                                           | 23        |
| <b>9</b>  | <b>Ocean Dynamics</b>                                                  | <b>23</b> |
| 9.1       | Boussinesq Hydrostatic Primitive Equations . . . . .                   | 23        |
| 9.1.1     | Tracer equations (skew-symmetric) . . . . .                            | 24        |
| 9.1.2     | Free-surface equation . . . . .                                        | 24        |
| 9.2       | Barotropic Solver (Forward–Backward, Semi-Implicit Coriolis) . . . . . | 24        |
| 9.3       | Equation of State (Wright 1997) . . . . .                              | 24        |
| 9.3.1     | Linear Equation of State . . . . .                                     | 24        |
| 9.4       | Hydrostatic Pressure and Vertical Velocity . . . . .                   | 25        |
| 9.5       | Ocean Vertical Mixing . . . . .                                        | 25        |
| 9.5.1     | K-Profile Parameterisation (KPP, Large et al. 1994) . . . . .          | 25        |
| 9.5.2     | Richardson-Number Mixing . . . . .                                     | 25        |
| 9.5.3     | Constant Mixing . . . . .                                              | 25        |
| 9.6       | Ocean Lateral Mixing . . . . .                                         | 25        |
| 9.6.1     | Harmonic (Laplacian) Diffusion . . . . .                               | 25        |
| 9.6.2     | Biharmonic ( $\nabla^4$ ) Diffusion . . . . .                          | 25        |
| 9.6.3     | Gent–McWilliams / Redi (GM/Redi) . . . . .                             | 25        |
| 9.7       | Ocean Bottom Drag . . . . .                                            | 26        |
| 9.8       | Ocean Surface Forcing . . . . .                                        | 26        |
| 9.9       | Shortwave Penetration . . . . .                                        | 26        |
| 9.10      | Ocean Convection . . . . .                                             | 26        |
| 9.10.1    | Enhanced Diffusion . . . . .                                           | 26        |
| 9.10.2    | Plume Model . . . . .                                                  | 26        |
| 9.11      | Ocean Conservation Fixers . . . . .                                    | 26        |
| 9.12      | Tracer Advection Schemes . . . . .                                     | 26        |
| 9.13      | Realistic-Geometry Spinup . . . . .                                    | 27        |
| <b>10</b> | <b>Time Integration</b>                                                | <b>27</b> |
| 10.1      | SSP-RK3 (Shu–Osher 1988) . . . . .                                     | 27        |
| 10.2      | Robert-Asselin-Williams (RAW) Time Filter (v3.7) . . . . .             | 27        |
| 10.3      | Adaptive Hyperdiffusion (v3.7) . . . . .                               | 27        |
| 10.4      | Split-Explicit Methods . . . . .                                       | 28        |
| <b>11</b> | <b>Land, Ice, and Coupler</b>                                          | <b>28</b> |
| 11.1      | Slab Land Model . . . . .                                              | 28        |
| 11.1.1    | Energy balance . . . . .                                               | 28        |
| 11.1.2    | Bucket hydrology . . . . .                                             | 28        |
| 11.2      | Sea-Ice Model . . . . .                                                | 28        |
| 11.2.1    | Surface temperature . . . . .                                          | 28        |
| 11.2.2    | Growth and melt . . . . .                                              | 28        |
| 11.2.3    | Concentration . . . . .                                                | 28        |
| 11.2.4    | EVP Rheology (Hunke & Dukowicz 1997) . . . . .                         | 28        |
| 11.2.5    | Multi-category ice . . . . .                                           | 29        |

|           |                                                                    |           |
|-----------|--------------------------------------------------------------------|-----------|
| 11.3      | Land Carbon Cycle and Stomatal Conductance . . . . .               | 29        |
| 11.3.1    | DALEC-990 Carbon Pools . . . . .                                   | 29        |
| 11.3.2    | Farquhar (1980) Photosynthesis . . . . .                           | 29        |
| 11.3.3    | Stomatal conductance . . . . .                                     | 29        |
| 11.4      | Plant Functional Types (PFTs) . . . . .                            | 29        |
| 11.5      | DifferLand Training . . . . .                                      | 29        |
| 11.6      | Tile Coupler . . . . .                                             | 30        |
| <b>12</b> | <b>Machine Learning: Spherical Fourier Neural Operator</b>         | <b>30</b> |
| 12.1      | Architecture . . . . .                                             | 30        |
| 12.2      | Spectral convolution . . . . .                                     | 31        |
| 12.3      | Neural and Learned Physics . . . . .                               | 31        |
| 12.3.1    | NeuralPhysics . . . . .                                            | 31        |
| 12.3.2    | LearnedColumn . . . . .                                            | 31        |
| <b>13</b> | <b>Conservation Corrections</b>                                    | <b>31</b> |
| 13.1      | Atmosphere . . . . .                                               | 31        |
| 13.2      | Ocean . . . . .                                                    | 31        |
| <b>14</b> | <b>Physical Constants</b>                                          | <b>32</b> |
| <b>15</b> | <b>Ocean Biogeochemistry</b>                                       | <b>32</b> |
| 15.1      | Carbonate Chemistry and Air-Sea CO <sub>2</sub> Exchange . . . . . | 32        |
| 15.2      | NPZD Ecosystem Model . . . . .                                     | 32        |
| <b>16</b> | <b>CMIP Infrastructure</b>                                         | <b>32</b> |
| 16.1      | CF/CMOR-Compliant Output . . . . .                                 | 32        |
| 16.2      | AMIP CMIP6 Forcing Deck . . . . .                                  | 33        |
| 16.3      | Experiment Templates . . . . .                                     | 33        |
| 16.4      | Restart and Reproducibility . . . . .                              | 33        |
| 16.5      | Tuning Guide . . . . .                                             | 33        |
| 16.6      | OMIP Simulation Pipeline . . . . .                                 | 34        |
| <b>17</b> | <b>4D-Var Data Assimilation (v3.9)</b>                             | <b>34</b> |
| 17.1      | Cost Function . . . . .                                            | 34        |
| 17.2      | Control Vector . . . . .                                           | 34        |
| 17.3      | Background Error Covariance . . . . .                              | 34        |
| 17.4      | Observation Operators . . . . .                                    | 34        |
| 17.5      | Minimiser . . . . .                                                | 35        |
| 17.6      | DA Cycling and Incremental Analysis . . . . .                      | 35        |
| <b>18</b> | <b>References</b>                                                  | <b>35</b> |

## Introduction

legoESM is a modular Earth System Model written entirely in JAX, supporting automatic differentiation through all components. The model couples atmospheric, oceanic, land, sea-ice, and biogeochemical components on the sphere, with 25+ dynamical cores spanning five discretisation families (C-D grid FV3, finite-volume PPM, pseudospectral, FC-Gram, C-grid) and 35+ physics parameterisations. The cubed-sphere implementation is unified around a single FV3-style C-D grid discretisation (Lin 2004, Putman & Lin 2007) shared by atmosphere (shallow water, hydrostatic PE, non-hydrostatic CE) and ocean, eliminating the Hollingsworth–Kallberg instability. The cubed-sphere transport uses an FV3-faithful PPM reconstruction with monotonicity limiters and flux-form mass transport on staggered grids, complemented by an FV3-exact forward–backward shallow-water core (`fv3_sw_core.py`) implementing the full `c_sw/d_sw` stepping and `d2a2c_vect` (D-grid  $\rightarrow$  A-grid  $\rightarrow$  C-grid) vector conversion with duo-grid sub-grid metrics ( $\sin \alpha_{sg}$ ,  $\cos \alpha_{sg}$ ) following Mouallem et al. (2023). The model includes a land carbon cycle with Farquhar photosynthesis and stomatal conductance, ocean biogeochemistry (abiotic carbon + NPZD ecosystem), sea-ice dynamics with EVP rheology and multi-category ice, CMIP6-class infrastructure (CF/CMOR output across four tables, experiment templates, restart reproducibility, documented tuning), and a fully differentiable 4D-Var data assimilation system leveraging JAX autodiff for adjoint-free gradient computation. The MPI-distributed runtime supports native 4D halo exchange (one message per neighbour for all vertical levels), reverse-mode AD through MPI via a custom `sendrecv` VJP wrapper, and AD-safe `allreduce(SUM)` for gradient-compatible global reductions. The codebase comprises approximately 385 source modules ( $\sim 102,000$  lines of code) validated by 3489 tests across 216 test files.

### Design principles.

- Pure-JAX implementation: all components are compatible with `jit`, `grad`, and `vmap`.
- Modular physics: each parameterisation follows a factory pattern returning a callable `physics_fn(state, grid, coord)  $\rightarrow$  Tendencities`.
- Multiple dynamical cores share the same physics interface, enabling interchangeable experiments.
- Neural-network components (SFNO) integrate seamlessly via the same JAX computational graph.

## Model Architecture

legoESM is organised as a *federation* of eight independently installable packages that all ship into one shared `legoesm` PEP-420 namespace, layered as an acyclic dependency graph whose import boundaries are enforced by zero-ignore `lint-imports` contracts (Fig. 1). A single `legoesm-core` substrate provides fields, grids, vertical coordinates, operators, the device/precision runtime, the parallel halo/reduction layer, and the grid  $\times$  complexity capability matrix; the four Earth-system components (`atmosphere`, `ocean`, `land`, `ice`) depend *only* on the substrate and never on each other, so each can be `pip` installed and gradient-checked in isolation. Above them sits an orchestration cluster — `coupler+driver`, `ml+training+da`, and `tools` — whose mutual edges form a uv-workspace cycle, so the three install together.

The “lego” in legoESM is the second design axis: standard tensor-in / tendency-out interfaces let every block — dynamical core, each physics parameterisation, grid, vertical coordinate, time integrator, and even classical-versus-neural physics — be swapped independently by a string scheme name resolved through a factory (Table 1). Figure 2 traces one coupled time step through these swap points.

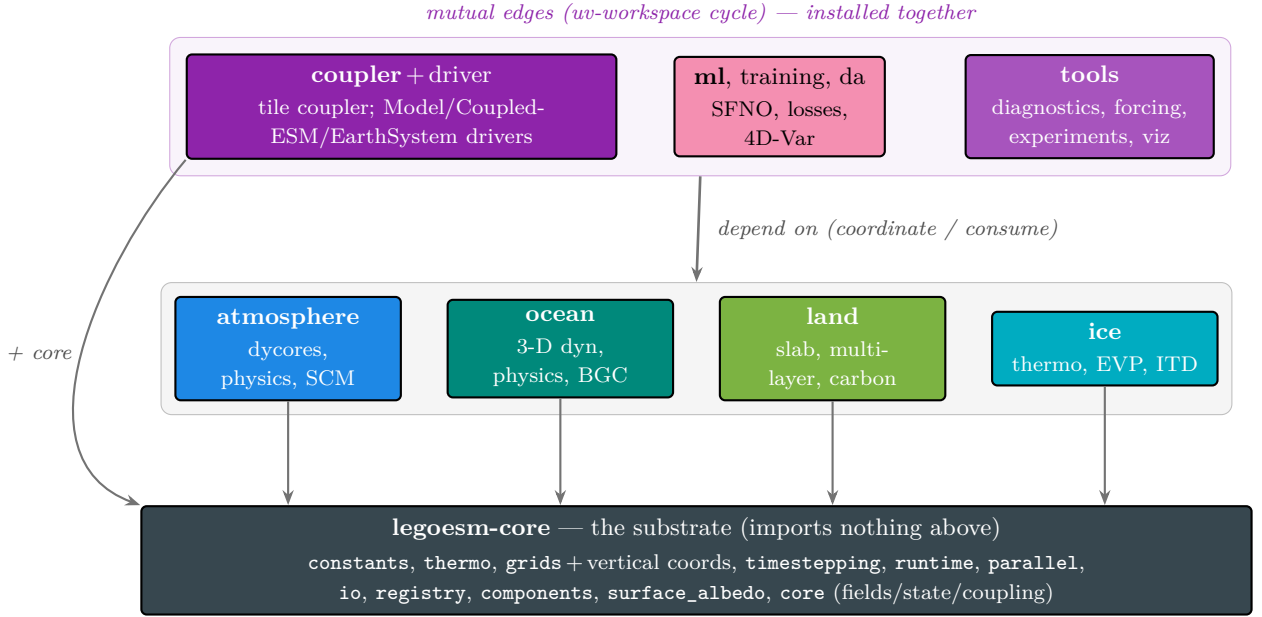


Figure 1: **Federation dependency DAG.** Arrows point from a package to what it imports (“depends on”); the layering is gated by zero-ignore import contracts (see `FEDERATION.md`). The four components are mutually independent and standalone-installable (contract #2); the orchestration cluster (coupler / ml / tools) sits above and carries the only mutual edges.

A coupled step runs at three nested cadences. The host *segment loop* (`build_segment_fn` → `run_segment_jit`) drives a `jax.lax.scan` over inner atmosphere steps; each inner step advances the dycore  $(u, v, T, p_s)$ , applies the gated dry-mass fixer, runs the unified physics pipeline (radiation sub-cycled by `lax.cond`; then convection, microphysics, surface exchange, turbulence/BL, and gravity-wave drag summed as tendencies), and closes with the moisture/energy state update. After each segment, `_segment_hook` sub-cycles the slow components at `coupling_dt` (default 3600s): the ocean step, the tile surface coupler (area-weighted bulk-flux blend over ocean/ice/land/lake), and the gated  $\text{CO}_2$  tracer, followed by conservation diagnostics. Because every block is a pure JAX pytree function and all communication is differentiable, the same graph supports forward simulation, `jit` compilation, and reverse-mode `grad`.

## Grid Systems

### Cubed-Sphere Grid

Six faces of a cube are mapped to the sphere via **gnomonic (central) projection**. Let  $(\alpha_x, \alpha_y) \in [-\pi/4, \pi/4]^2$  be gnomonic coordinates on a face.

#### Face-to-Cartesian mapping

For face 0 (+x face):

$$(x, y, z) = (1, \tan \alpha_x, \tan \alpha_y), \quad (1)$$

with analogous expressions for the remaining five faces. Points are normalised to the unit sphere:  $\hat{\mathbf{r}} = \mathbf{r}/|\mathbf{r}|$ , and geographic coordinates follow as  $\lambda = \text{atan2}(\hat{y}, \hat{x})$ ,  $\phi = \arcsin(\hat{z})$ .

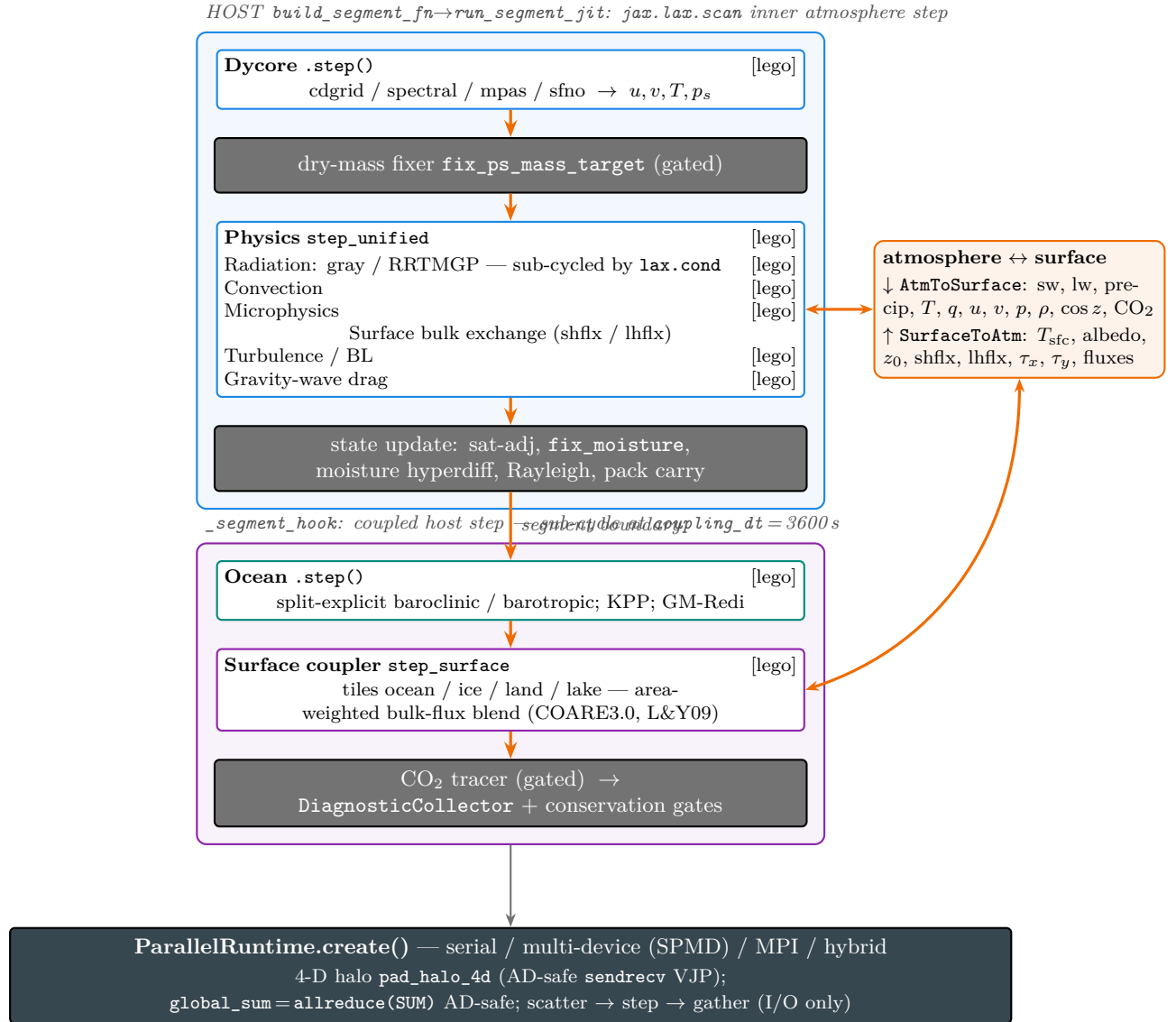


Figure 2: **Coupled time-step data flow.** White blocks tagged [lego] are factory-swapped by scheme name (Table 1); dark blocks are fixed numerics. The atmosphere inner step (blue) runs inside a `jax.lax.scan`; the slow components (purple) sub-cycle at `coupling_dt`. The orange link is the bidirectional atmosphere–surface flux exchange. The entire graph executes on one `ParallelRuntime` with AD-safe halo exchange and `allreduce(SUM)` reductions, so `jax.grad` flows end-to-end through the coupled step.

Table 1: **The “lego” swap dials.** Each axis is selected independently by a string scheme name resolved through the listed factory; an unknown name raises **ValueError**. Menus abbreviated.

| Swap axis             | Factory entry point                         | Example menu                                                                                                   |
|-----------------------|---------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Horizontal grid       | <code>create_grid()</code>                  | cubed-sphere, Gaussian/spectral, lat-lon, MPAS Voronoi, tripolar (ocean)                                       |
| Dynamical core        | <code>create_atmosphere_dycore()</code>     | {cdgrid, spectral, mpas, sfno} × {shallow water, hydrostatic PE, NH compressible Euler}; lat-lon C-grid; plane |
| Vertical coordinate   | <code>create_*_coordinate()</code>          | sigma, hybrid $\sigma$ - $p$ , height / $z^*$                                                                  |
| Time integrator       | <code>dispatch_integrator()</code>          | ssp_rk3, ssp_rk3_scan, ssp_rk34, ssp_rk54, rk4                                                                 |
| Radiation             | <code>make_radiation_physics()</code>       | gray (Frierson), RRTMG correlated- $k$                                                                         |
| Convection            | <code>make_convection_physics()</code>      | Bechtold, Tiedtke, Kain-Fritsch, Zhang-McFarlane, Emanuel, Kuo, SBM                                            |
| Microphysics          | <code>make_microphysics_physics()</code>    | Kessler, Sundqvist, Morrison, Seifert-Beheng, Thompson, ML                                                     |
| Turbulence / BL       | <code>make_turbulence_physics()</code>      | Louis, Holtslag-Boville, YSU, EDMF, TKE, CLUBB-lite, Smagorinsky                                               |
| Gravity-wave drag     | <code>make_gwd_physics()</code>             | Lindzen, Hines, McFarlane, Rayleigh, spectral, ML                                                              |
| Physics core          | <code>make_{physics,hybrid,neural}</code>   | classical, hybrid, neural (SFNO)                                                                               |
| Ocean vertical mixing | <code>make_vertical_mixing_physics()</code> | (KPP, Richardson, constant, Jayne-St-Laurent tidal                                                             |
| Ocean lateral mixing  | <code>make_lateral_mixing_physics()</code>  | harmonic, biharmonic, GM-Redi, Visbeck, Leith, backscatter                                                     |
| Ocean surface forcing | <code>make_surface_forcing_physics()</code> | (bulk, restoring, prescribed                                                                                   |
| Complexity ladder     | <code>resolve_model_complexity()</code>     | idealized, intermediate, full                                                                                  |

## Cell area

The area element on the gnomonic equidistant grid is

$$\Delta A = \frac{R^2 \Delta \alpha^2}{\cos^2 \alpha_x \cos^2 \alpha_y D^3}, \quad D = \sqrt{1 + \tan^2 \alpha_x + \tan^2 \alpha_y}, \quad \Delta \alpha = \frac{\pi}{2n}. \quad (2)$$

## Grid spacing

Great-circle distances use the chord-to-arc formula:

$$d = R \cdot 2 \arcsin\left(\frac{|\mathbf{r}_1 - \mathbf{r}_2|}{2}\right). \quad (3)$$

Metric factors  $\Delta x_{i,j}$  and  $\Delta y_{i,j}$  are computed as two-cell spans for second-order centred differences.

## Grid angle and wind rotation

The angle  $\theta$  between the grid  $x$ -axis and geographic east is

$$\theta = \text{atan2}\left(\frac{\partial \phi}{\partial i}, \frac{\partial \lambda}{\partial i} \cos \phi\right). \quad (4)$$

Wind rotation between geographic  $(u_E, v_N)$  and grid-aligned  $(u_g, v_g)$ :

$$\begin{pmatrix} u_E \\ v_N \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} u_g \\ v_g \end{pmatrix}. \quad (5)$$

## Halo exchange

Scalar fields are exchanged directly across the six-face connectivity table. Vector fields are first rotated to geographic components, padded as scalars, then rotated back using the padded grid angle.

Two halo widths are supported. **Halo=1** (one ghost cell per side) suffices for second-order centred differences and MUSCL reconstruction. **Halo=2** (two ghost cells per side) is required by higher-order reconstruction stencils (PPM, WENO5). The halo exchange interpolates neighbour-face strips to the correct gnomonic position at each depth, removing the  $O(\Delta\alpha)$  mismatch near cube corners.

Under MPI, the native **4D halo exchange** path (`pad_halo_4d`, `pad_halo_vector_4d`) exchanges all vertical levels in a single MPI message per neighbour, replacing the earlier per-level `vmap(pad_halo)` path. All `sendrecv` calls go through a `@jax.custom_vjp` wrapper that swaps source/dest in the backward pass, enabling `jax.grad` through distributed halo exchange for differentiable MPI simulations.

The May 2026 GPU-scaling branch (#228) further compresses per-RK3-step collectives across all dycore backends: cubed-sphere allreduces fuse  $4 \rightarrow 1$  (halo bandwidth reduced  $\sim 36\text{--}144\times$  at high resolution), the spectral PE FFT and Legendre transforms fuse into a single pass with stacked per-tracer SH analyses, and MPAS `sendrecvs` drop  $12 \rightarrow 6$  with mass-fix payloads  $3 \rightarrow 2$ . Fifteen SPMD-correctness unit tests gate the changes. Real-hardware Levante-cluster validation is described in `docs/REAL_HARDWARE_SCALING.md` and `docs/GPU_SCALING_BRANCH.md`.

## Gaussian Grid and Spectral Transforms

The spectral dynamical cores use a Gaussian grid with triangular truncation at total wavenumber  $n_{\max}$ .

### Grid dimensions

$$n_{\text{lat}} = \lceil \frac{3}{2}(n_{\max} + 1) \rceil_{\text{even}}, \quad n_{\text{lon}} = 2 n_{\text{lat}}, \quad n_{\text{sh}} = \frac{(n_{\max} + 1)(n_{\max} + 2)}{2}. \quad (6)$$

Latitudes are Gaussian quadrature points  $\mu_k = \sin \phi_k$  with weights  $w_k$ ; longitudes are equispaced.

### Associated Legendre polynomials

Fully normalised  $(4\pi)$  polynomials  $P_n^m(\mu)$  satisfy  $\int |Y_n^m|^2 dA = 1$ . They are computed via the recursion

$$P_m^m = -\sqrt{\frac{2m+1}{2m}} \sqrt{1-\mu^2} P_{m-1}^{m-1}, \quad (7)$$

$$P_n^m = \sqrt{\frac{4n^2-1}{n^2-m^2}} \left( \mu P_{n-1}^m - \sqrt{\frac{(n-1)^2-m^2}{4(n-1)^2-1}} P_{n-2}^m \right). \quad (8)$$

The derivative operator  $H_n^m = -(1-\mu^2) dP_n^m/d\mu$  is used for meridional gradients.

### Spherical harmonic transforms

**Analysis (grid  $\rightarrow$  spectral).**

$$\hat{f}_{nm} = 2\pi \sum_k w_k P_n^m(\mu_k) \tilde{f}_m(\mu_k), \quad \tilde{f}_m = \frac{1}{n_{\text{lon}}} \text{FFT}[f]_m. \quad (9)$$

### Synthesis (spectral $\rightarrow$ grid).

$$f(\phi, \lambda) = \text{iFFT} \left[ \sum_{n=m}^{n_{\max}} \hat{f}_{nm} P_n^m(\mu) \right]. \quad (10)$$

### Spectral operators

Laplacian eigenvalue:  $\nabla^2 \hat{f}_{nm} = -n(n+1)/a^2 \cdot \hat{f}_{nm}$ . Inverse Laplacian:  $\nabla^{-2} \hat{f}_{nm} = -a^2/[n(n+1)]$  for  $n > 0$ . Hyperdiffusion:  $-\nu[n(n+1)/a^2]^p \hat{f}_{nm}$ .

### Velocity from vorticity and divergence

Streamfunction  $\psi = \nabla^{-2} \zeta$ , velocity potential  $\chi = \nabla^{-2} \delta$ . Velocity reconstruction:

$$u \cos \phi = \frac{\partial \psi}{\partial \theta} + \frac{\partial \chi}{\partial \lambda}, \quad v \cos \phi = \frac{\partial \psi}{\partial \lambda} - \frac{\partial \chi}{\partial \theta}, \quad (11)$$

where  $\theta$  is colatitude.

### Sigma Vertical Coordinate

$$\sigma = p / p_s, \quad \sigma \in [\sigma_{\text{top}}, 1]. \quad (12)$$

Half-level interfaces  $\sigma_{k+1/2}$  are uniformly spaced; full levels  $\sigma_k = (\sigma_{k-1/2} + \sigma_{k+1/2})/2$ . Layer thickness  $\Delta\sigma_k = \sigma_{k+1/2} - \sigma_{k-1/2}$ .

### Simmons–Burridge geopotential

The geopotential at full level  $k$  is

$$\Phi_k = \Phi_s + R_d \sum_{k'=k+1}^{N-1} T_{k'} \ln \frac{\sigma_{k'+1/2}}{\sigma_{k'-1/2}} + \alpha_k R_d T_k, \quad (13)$$

where the Simmons–Burridge coefficient is

$$\alpha_k = 1 - \frac{\sigma_{k-1/2}}{\Delta\sigma_k} \ln \frac{\sigma_{k+1/2}}{\sigma_{k-1/2}}. \quad (14)$$

### Sigma-dot

The vertical velocity in  $\sigma$ -space is obtained by integrating the continuity equation from the model top ( $\dot{\sigma} = 0$ ):

$$\dot{\sigma}_{k+1/2} = \frac{\sigma_{k+1/2} - \sigma_{\text{top}}}{1 - \sigma_{\text{top}}} D_{\text{tot}} - \sum_{k'=0}^k \nabla \cdot \mathbf{v}_{k'} \Delta\sigma_{k'}, \quad D_{\text{tot}} = \sum_{k'=0}^{N-1} \nabla \cdot \mathbf{v}_{k'} \Delta\sigma_{k'}. \quad (15)$$

### Pressure velocity

Since  $p = \sigma p_s$ , the material pressure tendency is

$$\omega \equiv \frac{dp}{dt} = \sigma \frac{dp_s}{dt} + p_s \dot{\sigma}, \quad (16)$$

where  $dp_s/dt$  is the *material* derivative of surface pressure (Eulerian tendency plus advection by the horizontal wind).

## Height and $z^*$ Coordinates

The terrain-following  $z^*$  coordinate:

$$z^* = H \frac{z - z_s}{H - z_s}, \quad J = \frac{\partial z}{\partial z^*} = \frac{H - z_s}{H}. \quad (17)$$

Physical height:  $z(z^*, x, y) = z_s + z^*(H - z_s)/H$ .

For the **ocean**, the  $z^*$  coordinate has a dynamic Jacobian:

$$J(\eta) = \frac{\eta + H}{H_{\max}}, \quad (18)$$

where  $\eta$  is the free-surface elevation and  $H$  the resting depth.

## Core Operators

All finite-difference operators use second-order centred differences on the A-grid (collocated) with halo padding on the cubed sphere. The discrete difference operators  $\delta_i$  and  $\delta_j$  are defined as centred differences of *cell-centre* values on the padded grid:

$$\delta_i[q]_{i,j} = q_{i+1,j} - q_{i-1,j}, \quad \delta_j[q]_{i,j} = q_{i,j+1} - q_{i,j-1}. \quad (19)$$

All metric-weighted fluxes ( $u h_y$ ,  $v h_x$ ) are evaluated at cell centres before differencing, consistent with the A-grid stencil.

### Gradient

$$\left. \frac{\partial f}{\partial x} \right|_{i,j} = \frac{f_{i+1,j} - f_{i-1,j}}{2 \Delta x_{i,j}}, \quad \left. \frac{\partial f}{\partial y} \right|_{i,j} = \frac{f_{i,j+1} - f_{i,j-1}}{2 \Delta y_{i,j}}. \quad (20)$$

### Divergence (flux form)

$$\nabla \cdot (u, v) = \frac{1}{2 A_{i,j}} \left[ \delta_i(u h_y) + \delta_j(v h_x) \right], \quad (21)$$

where  $h_x = \Delta x/2$ ,  $h_y = \Delta y/2$  are half-edge lengths and  $A$  is the cell area.

### Vorticity

$$\zeta = \frac{1}{2 A_{i,j}} \left[ \delta_i(v h_y) - \delta_j(u h_x) \right]. \quad (22)$$

## C-D Grid Operators (FV3-style, `operators_cdgrid.py`)

The canonical cubed-sphere operator module uses FV3-style C-D grid staggering (Lin 2004, Putman & Lin 2007), shared by all atmosphere (SW, PE, CE) and ocean dynamical cores. D-grid winds at cell corners ( $6, n+1, n+1$ ) are prognostic; C-grid velocities at cell edges are diagnosed for mass/scalar transport.

**FV3-faithful PPM transport (v3.9).** The operator module was rewritten ( $\sim 740$  lines) with proper PPM reconstruction, Colella–Woodward monotonicity limiters, and flux-form mass transport on staggered grids, faithfully following the FV3 algorithm of Putman & Lin (2007). This replaces the earlier simplified A-grid transport with a fully staggered C-D grid transport that correctly handles cross-face halo exchange, geographic-to-grid wind rotation, and corner interpolation for the cubed sphere.

**D-grid vorticity.** Vorticity at each cell corner is the discrete circulation integral divided by the corner dual-cell area  $A_c$ :

$$\zeta_{\text{corner}} = \frac{1}{A_c} \oint \mathbf{u} \cdot d\mathbf{l} \approx \frac{1}{A_c} [u_{i,j}^D \Delta x - u_{i+1,j}^D \Delta x + v_{i,j+1}^D \Delta y - v_{i,j}^D \Delta y]. \quad (23)$$

This circulation-based discretisation is exact on the D-grid and eliminates the Hollingsworth–Kallberg instability that plagues A-grid vorticity operators.

**Arakawa–Lamb gradient.** The Bernoulli (or Exner) gradient at D-grid corners is a 4-point Arakawa–Lamb average:

$$\left. \frac{\partial B}{\partial x} \right|_{i+\frac{1}{2},j+\frac{1}{2}} = \frac{B_{i+1,j} + B_{i+1,j+1} - B_{i,j} - B_{i,j+1}}{2\Delta x}, \quad (24)$$

and analogously for  $\partial B/\partial y$ .

**C-grid mass-flux divergence.** Scalar transport uses upwind C-grid mass fluxes:

$$\frac{dh}{dt} = -\frac{1}{A} [(hu_c)_{i+\frac{1}{2},j} - (hu_c)_{i-\frac{1}{2},j} + (hv_c)_{i,j+\frac{1}{2}} - (hv_c)_{i,j-\frac{1}{2}}], \quad (25)$$

where  $u_c, v_c$  are diagnosed from  $u_d, v_d$  by face-edge averaging.

**FV3 forward–backward shallow-water core (fv3\_sw\_core.py).** An FV3-exact implementation of the complete `c_sw/d_sw` forward–backward stepping algorithm (Lin 2004). The forward step (`c_sw`) computes C-grid mass fluxes and height tendencies; the backward step (`d_sw`) updates D-grid winds using the vorticity flux and pressure gradient. Key components:

- **d2a2c\_vect:** D-grid  $\rightarrow$  A-grid  $\rightarrow$  C-grid vector conversion, faithfully porting the GFDL FV3 algorithm including edge-boundary stencils, one-sided interpolation near cube edges, and non-orthogonality corrections via  $\cos \alpha / \sin^{-2} \alpha$  metrics.
- **PPM wind advection:** Piecewise Parabolic Method reconstruction for C-grid wind tendencies (`xtp_u, ytp_v`), with FV3-specific interpolation coefficients ( $A_1, A_2, C_1-C_3$ ).
- **PPM vorticity transport:** Monotonic PPM reconstruction for the absolute vorticity flux, stabilising the D-grid wind update.
- **Kinetic energy:** Cell-centred kinetic energy computed from A-grid velocities with optional 4th-order averaging via `a2b_ord4`.

**Duo-grid sub-grid metrics (Mouallem et al. 2023).** The cubed-sphere C-D grid includes 9-point sub-grid angle metrics per cell, stored as  $\sin \alpha_{\text{sg}}$  and  $\cos \alpha_{\text{sg}}$  arrays of shape  $(6, n, n, 9)$ . These encode the local deviation from orthogonality at cell centres, edge midpoints, and corners, enabling FV3-faithful flux scaling at cube-face boundaries where the gnomonic projection introduces non-orthogonality. Additional C-D grid metrics include centre-to-centre distances  $\Delta x_c, \Delta y_c$ , and non-orthogonality factors  $\cos \alpha_{\text{corner}}, (\sin^2 \alpha_{\text{corner}})^{-1}$  at D-grid corners.

## Laplacian and Hyperdiffusion

$$\nabla^2 f = \nabla \cdot (\nabla f), \quad \text{Hyperdiffusion:} \quad -\nu \nabla^4 f = -\nu \nabla^2 (\nabla^2 f). \quad (26)$$

## Finite-Volume Transport

The finite-volume dynamical cores use upwind reconstruction at cell edges coupled with a Rusanov approximate Riemann solver. Three reconstruction methods are available, selected via the `limiter` parameter.

### MUSCL Reconstruction (2nd order, halo = 1)

Piecewise-linear reconstruction with slope limiting (van Leer, 1979). At the interface between cells  $i$  and  $i+1$ :

$$q_L = q_i + \frac{1}{2} \phi_i, \quad q_R = q_{i+1} - \frac{1}{2} \phi_{i+1}, \quad (27)$$

where  $\phi_i$  is the limited slope. Two limiters are provided:

- **Minmod**:  $\phi = \minmod(\Delta q_+, \Delta q_-)$ ,
- **MC** (monotonised central):  $\phi = \minmod(\minmod(2\Delta q_+, 2\Delta q_-), \frac{1}{2}(\Delta q_+ + \Delta q_-))$ .

### PPM Reconstruction (4th order, halo = 2)

The Piecewise Parabolic Method (Colella & Woodward, 1984) constructs a monotone parabola in each cell from two edge values and the cell average.

#### Fourth-order edge values

At the interface  $i + \frac{1}{2}$ :

$$\hat{q}_{i+1/2} = \frac{7}{12}(q_i + q_{i+1}) - \frac{1}{12}(q_{i-1} + q_{i+2}). \quad (28)$$

#### Monotonicity constraint

The edge value is clamped to lie between its two adjacent cell averages:

$$\hat{q}_{i+1/2} \leftarrow \text{median}(\hat{q}_{i+1/2}, q_i, q_{i+1}). \quad (29)$$

#### Colella–Woodward limiting

For each cell  $i$  with left edge value  $a_L = \hat{q}_{i-1/2}$  and right edge value  $a_R = \hat{q}_{i+1/2}$ :

1. If  $(a_R - q_i)(q_i - a_L) \leq 0$  (local extremum): set  $a_L = a_R = q_i$ .
2. Define  $\delta_m = a_R - a_L$  and  $\delta_6 = 6(q_i - \frac{1}{2}(a_L + a_R))$ .
  - If  $\delta_m \delta_6 > \delta_m^2$ : set  $a_L = 3q_i - 2a_R$ .
  - If  $\delta_m \delta_6 < -\delta_m^2$ : set  $a_R = 3q_i - 2a_L$ .

The left and right states at each edge are then  $q_L = a_R$  of the cell to the left and  $q_R = a_L$  of the cell to the right.

### WENO5 Reconstruction (5th order, halo = 2)

The 5th-order Weighted Essentially Non-Oscillatory scheme (Jiang & Shu, 1996) combines three 3rd-order sub-stencil polynomials with nonlinear weights that adapt to local smoothness.

#### Left-biased reconstruction at $i + \frac{1}{2}$

Using cells  $q_{i-2}, \dots, q_{i+2}$ :

$$q^{(0)} = \frac{1}{6}(2q_{i-2} - 7q_{i-1} + 11q_i), \quad d_0 = \frac{1}{10}, \quad (30)$$

$$q^{(1)} = \frac{1}{6}(-q_{i-1} + 5q_i + 2q_{i+1}), \quad d_1 = \frac{6}{10}, \quad (31)$$

$$q^{(2)} = \frac{1}{6}(2q_i + 5q_{i+1} - q_{i+2}), \quad d_2 = \frac{3}{10}. \quad (32)$$

#### Smoothness indicators

$$\beta_0 = \frac{13}{12}(q_{i-2} - 2q_{i-1} + q_i)^2 + \frac{1}{4}(q_{i-2} - 4q_{i-1} + 3q_i)^2, \quad (33)$$

$$\beta_1 = \frac{13}{12}(q_{i-1} - 2q_i + q_{i+1})^2 + \frac{1}{4}(q_{i-1} - q_{i+1})^2, \quad (34)$$

$$\beta_2 = \frac{13}{12}(q_i - 2q_{i+1} + q_{i+2})^2 + \frac{1}{4}(3q_i - 4q_{i+1} + q_{i+2})^2. \quad (35)$$

### Nonlinear weights

$$\alpha_k = \frac{d_k}{(\varepsilon + \beta_k)^2}, \quad \omega_k = \frac{\alpha_k}{\sum_j \alpha_j}, \quad q_L = \sum_k \omega_k q^{(k)}, \quad \varepsilon = 10^{-6}. \quad (36)$$

### Right-biased reconstruction

The right state  $q_R$  at  $i + \frac{1}{2}$  is obtained by mirroring the stencil around the interface, using cells  $q_{i-1}, \dots, q_{i+3}$  with ideal weights  $d_0 = \frac{1}{10}$ ,  $d_1 = \frac{6}{10}$ ,  $d_2 = \frac{3}{10}$  applied to the reversed sub-stencils.

### Rusanov Flux

At each edge, the mass flux is

$$F = \frac{1}{2}(h_L u_L + h_R u_R) - \frac{1}{2} \alpha (h_R - h_L), \quad \alpha = \max(|u_L| + c_L, |u_R| + c_R), \quad (37)$$

where  $c = \sqrt{gh}$  is the gravity-wave speed (set to zero for pure transport without wave-speed dissipation). The net flux divergence for cell  $i$  is

$$-(\nabla \cdot \mathbf{F})_i = -\frac{1}{A_i} \left[ (F_x h_y)_{i+1/2} - (F_x h_y)_{i-1/2} + (G_y h_x)_{j+1/2} - (G_y h_x)_{j-1/2} \right], \quad (38)$$

where  $h_x, h_y$  are the edge metric factors.

### Well-balanced scheme

When topography  $h_s$  is present, the scheme reconstructs the free surface  $\eta = h + h_s$  instead of  $h$ . The fluid depth at each edge is recovered as  $h_L = \max(\eta_L - h_{s,\text{edge}}, 0)$ , ensuring zero spurious flux for a lake-at-rest state (Audusse et al., 2004).

## FC-Gram Spectral Operators

The Fourier Continuation (FC-Gram) method extends periodic Fourier spectral methods to non-periodic domains by appending a smooth continuation interval. On each cubed-sphere face, the 1D function  $f(x)$  on  $[0, 1]$  is extended to a periodic function on  $[0, 1 + d]$  via a Gram polynomial basis in the continuation region.

### Extension operator

Given  $f$  sampled at  $n$  equispaced points, the extension  $f_{\text{ext}}$  is constructed as

$$f_{\text{ext}}(x) = \sum_{k=0}^{K-1} \alpha_k G_k(x), \quad x \in [1, 1 + d], \quad (39)$$

where  $G_k$  are orthogonal Gram polynomials and the coefficients  $\alpha_k$  are chosen so that  $f_{\text{ext}}$  smoothly matches  $f$  at both endpoints, yielding a globally smooth periodic extension.

### Spectral differentiation

On the extended periodic domain of length  $L_{\text{ext}} = 1 + d$ , differentiation is performed via FFT:

$$f'_j = \text{IFFT} \left[ i k \text{FFT}[f_{\text{ext}}] \right] \Big|_{x_j \in [0, 1]}, \quad (40)$$

where  $k$  are the Fourier wavenumbers on the extended domain. This achieves spectral accuracy in the interior with controlled Gibbs artifacts at face boundaries.

## C-grid-style divergence damping

The FC-Gram variants with divergence damping add 2nd and 4th order selective dissipation of divergent modes:

$$D_{\text{div}} = -\nu_2 \nabla(\nabla \cdot \mathbf{v}) - \nu_4 \nabla(\nabla^2(\nabla \cdot \mathbf{v})), \quad (41)$$

applied to the momentum equations. This selectively damps gravity-wave modes while preserving rotational (balanced) flow, mirroring the divergence damping strategy used in FV3 on its C-D grid.

## Atmospheric Dynamics

### Shallow Water Equations

In **vector-invariant form** on the cubed sphere:

$$\frac{\partial h}{\partial t} = -\nabla \cdot (h \mathbf{v}), \quad (42)$$

$$\frac{\partial u}{\partial t} = (\zeta + f) v - \frac{\partial B}{\partial x} + D_u, \quad (43)$$

$$\frac{\partial v}{\partial t} = -(\zeta + f) u - \frac{\partial B}{\partial y} + D_v, \quad (44)$$

where the Bernoulli function is  $B = \frac{1}{2}(u^2 + v^2) + g(h + h_s)$  and  $D_u, D_v$  represent hyperdiffusion.

The mass equation (42) is discretised using the finite-volume Rusanov flux (Section 5) with a configurable reconstruction method: MUSCL (2nd order), PPM (4th order), or WENO5 (5th order), selected via `ShallowWaterConfig.fv_limiter`. Momentum uses centred differences with velocity-only biharmonic hyperdiffusion.

### Hydrostatic Primitive Equations ( $\sigma$ -coordinates)

Prognostic variables:  $u, v$  (winds),  $T$  (temperature),  $p_s$  (surface pressure).

$$\frac{\partial p_s}{\partial t} = - \int_0^1 \nabla \cdot (p_s \mathbf{v}) d\sigma, \quad (45)$$

$$\frac{\partial u}{\partial t} = (\zeta + f) v - \frac{\partial B}{\partial x} - R_d T \frac{\partial \ln p_s}{\partial x} + D_u, \quad (46)$$

$$\frac{\partial v}{\partial t} = -(\zeta + f) u - \frac{\partial B}{\partial y} - R_d T \frac{\partial \ln p_s}{\partial y} + D_v, \quad (47)$$

$$\frac{\partial T}{\partial t} = -\nabla \cdot (T \mathbf{v}) + T \nabla \cdot \mathbf{v} - \dot{\sigma} \frac{\partial T}{\partial \sigma} + \kappa T \frac{\omega}{p} + Q, \quad (48)$$

where  $B = K + \Phi$ ,  $K = \frac{1}{2}(u^2 + v^2)$ , and  $\kappa = R_d/c_p$ .

## Spectral Primitive Equations (Vorticity–Divergence)

On the Gaussian grid, the momentum equations are recast in the vorticity–divergence form (Bourke, 1972):

$$\frac{\partial \hat{\zeta}}{\partial t} = \nabla \times [\widehat{[(\zeta + f)\mathbf{v}]}] + \widehat{\text{vert. adv.}} - \nu \nabla^{2p} \hat{\zeta}, \quad (49)$$

$$\frac{\partial \hat{\delta}}{\partial t} = \nabla \cdot [\widehat{[(\zeta + f)\mathbf{v}]}] - \nabla^2 (K + \Phi + \widehat{R_d T \ln p_s}) + \widehat{\text{vert. adv.}} - \nu \nabla^{2p} \hat{\delta}, \quad (50)$$

$$\frac{\partial \hat{T}}{\partial t} = -\nabla \cdot (\widehat{T\mathbf{v}}) + T \nabla \cdot \mathbf{v} - \dot{\sigma} \widehat{\partial T / \partial \sigma} + \kappa \widehat{T\omega/p}, \quad (51)$$

$$\frac{\partial \widehat{\ln p_s}}{\partial t} = -\int_0^1 \hat{\delta} \, d\sigma. \quad (52)$$

The spectral curl and divergence of flux vectors  $(U, V) = (F_u \cos \phi, F_v \cos \phi)$  are

$$\widehat{\text{curl}} = \frac{im}{a} \mathcal{S}[V / \cos^2 \phi] + \frac{1}{a} \mathcal{S}_\mu[U], \quad \widehat{\text{div}} = \frac{im}{a} \mathcal{S}[U / \cos^2 \phi] - \frac{1}{a} \mathcal{S}_\mu[V], \quad (53)$$

where  $\mathcal{S}$  and  $\mathcal{S}_\mu$  denote the SH analysis with standard and  $dP_n^m/d\mu$  Legendre matrices.

## Non-Hydrostatic Compressible Euler Equations

In the  $z^*$  terrain-following coordinate, the prognostic variables are  $u, v, w, \theta', \rho'$  (perturbations from a reference state  $\rho_0, \theta_0, \pi_0$ ).

$$\frac{\partial u}{\partial t} = (\zeta + f) v - \frac{\partial K}{\partial x} - c_p \theta \frac{\partial \pi'}{\partial x} + D_u - r u, \quad (54)$$

$$\frac{\partial v}{\partial t} = -(\zeta + f) u - \frac{\partial K}{\partial y} - c_p \theta \frac{\partial \pi'}{\partial y} + D_v - r v, \quad (55)$$

$$\frac{\partial w}{\partial t} = -\frac{c_p \theta}{J} \frac{\partial \pi'}{\partial z^*} - g \frac{\theta'}{\theta_0} - r w, \quad (56)$$

$$\frac{\partial \theta'}{\partial t} = -\nabla \cdot (\theta \mathbf{v}_h) - \frac{w}{J} \frac{\partial \theta}{\partial z^*} + D_\theta, \quad (57)$$

$$\frac{\partial \rho'}{\partial t} = -\frac{1}{J} \left[ \nabla_h \cdot (J \rho \mathbf{v}_h) + \frac{\partial(\rho w)}{\partial z^*} \right], \quad (58)$$

where  $r(z)$  is the sponge-layer Rayleigh damping coefficient and the Exner function perturbation is

$$\pi' = \pi_0 \left[ \left( 1 + \frac{\rho'}{\rho_0} \right) \left( 1 + \frac{\theta'}{\theta_0} \right) \right]^{R_d/c_v} - \pi_0. \quad (59)$$

## Split-explicit time integration

The equations are integrated with an outer SSP-RK3 scheme for the slow tendencies (advection, Coriolis, horizontal pressure gradient, diffusion) and inner forward–backward acoustic substeps for the fast variables  $(w, \theta', \rho')$ .

### Forward step (w).

$$w^{n+1} = w^n + \Delta t_s \left[ -\frac{c_p \theta}{J} \frac{\partial \pi'}{\partial z^*} - g \frac{\theta'}{\theta_0} \right]. \quad (60)$$

### Backward step ( $\rho', \theta'$ ).

$$\rho'^{n+1} = \rho'^n - \Delta t_s \frac{\partial(\rho w^{n+1})}{\partial z^*}, \quad \theta'^{n+1} = \theta'^n - \Delta t_s \frac{w^{n+1}}{J} \frac{\partial \theta}{\partial z^*}. \quad (61)$$

An optional **semi-implicit acoustic** solve replaces the forward Euler for  $w$  with a tridiagonal system to remove the vertical acoustic CFL constraint.

**Acoustic off-centering (v3.7).** To damp vertically propagating acoustic/gravity wave noise in long integrations, the density update can be off-centred (Skamarock & Klemp 2008):

$$\rho_\beta'^{n+1} = (1 + \beta) \rho'^{n+1} - \beta \rho'^n, \quad (62)$$

where  $\beta \geq 0$  is the off-centering parameter (`acoustic_off_centering` in the config). For  $\beta = 0$  the scheme is centred (neutral);  $\beta = 0.1$  is recommended for AMIP/CMIP runs. This adds a small amount of temporal diffusion to the fast modes without tightening the horizontal CFL constraint.

### Sponge layer

$$r(z) = r_{\max} \sin^2\left(\frac{\pi}{2} \frac{\max(0, z - z_{\text{bot}})}{z_{\text{top}} - z_{\text{bot}}}\right). \quad (63)$$

### Turbulence closure: CRM, LES, and DNS

The diffusion terms  $D_u, D_v, D_w, D_\theta$  are evaluated in flux form,  $D_\phi = \rho_0^{-1} \partial_j(\rho_0 K_\phi \partial_j \phi)$ , over the horizontal directions and (optionally) a mass-weighted vertical leg. The same discrete operator spans three turbulence-resolving regimes that differ only in the viscosity  $K_m$  and the grid spacing  $\Delta x$  (heat and scalars use  $K_h = K_m/\text{Pr}$ ), selected by the `turbulence_closure` parameter:

- **CRM** (convection-permitting,  $\Delta x \sim 1\text{--}4\text{ km}$ ; `smagorinsky`): the 3-D Smagorinsky–Lilly eddy viscosity  $K_m = (C_s \Delta)^2 \sqrt{\max(0, |S|^2 - \text{Pr} N^2)}$  with  $\Delta = (\Delta x \Delta y \Delta z)^{1/3}$ , the full strain  $|S|^2 = 2S_{ij}S_{ij}$ ,  $C_s \approx 0.19$ , and the SAM `dosmagor` moist stratification cutoff  $N^2$ . The energy-containing eddies are entirely sub-grid; this is the regime validated against gSAM for the GATE, LBA, and RCE cases.
- **LES** (large-eddy simulation,  $\Delta x \sim 10\text{--}100\text{ m}$ ; `smagorinsky`): the same closure with  $C_s \approx 0.15$  and an optional near-wall limiter  $\ell_m = \min(C_s \Delta, \kappa z)$ ; the energy-containing eddies are resolved. gSAM’s GATE and LBA references are themselves 100 m LES, so this mode uses the identical closure at the identical resolution.
- **DNS** (direct numerical simulation,  $\Delta x \rightarrow$  the Kolmogorov scale; `molecular`): no turbulence model;  $K_m$  is the constant molecular kinematic viscosity  $\nu$  with  $K_h = \nu/\text{Pr}$  ( $\text{Pr} \approx 0.71$  for air), giving the molecular dissipation  $\nu \nabla^2 \mathbf{u}$  directly.

Because the three regimes share one discrete flux-form operator, LES and DNS are configuration changes rather than separate code paths; the molecular closure reproduces the analytical viscous decay  $\partial_t \hat{u} = -\nu k^2 \hat{u}$  of an isolated shear mode to within the time-integration error.

## Atmospheric Physics

### Gray Radiation (Frierson 2006)

#### Longwave optical depth

$$\tau(\sigma, \phi) = [f_l \sigma + (1 - f_l) \sigma^4] [\tau_e + (\tau_p - \tau_e) \sin^2 \phi] + \sum_k \tau_{\text{moist}} q_{v,k} \frac{\Delta p_k}{g}, \quad (64)$$

where the moisture optical depth per layer is  $\Delta\tau_{\text{moist},k} = \tau_{\text{moist}} q_{v,k} \Delta p_k / g$ , with  $\tau_e = 7.2$ ,  $\tau_p = 1.8$ ,  $f_l = 0.2$ ,  $\tau_{\text{moist}} = 0.0115 \text{ m}^2 \text{ kg}^{-1}$ .

### Two-stream LW fluxes

Layer transmittance and emissivity:

$$\mathcal{T}_k = e^{-D \Delta\tau_k}, \quad \varepsilon_k = 1 - \mathcal{T}_k, \quad D = 1.66. \quad (65)$$

Upward sweep from surface ( $F_{\text{sfc}}^\uparrow = \varepsilon_s \sigma_{\text{SB}} T_s^4$ ):

$$F_k^\uparrow = \mathcal{T}_k F_{k+1}^\uparrow + \varepsilon_k \sigma_{\text{SB}} T_k^4. \quad (66)$$

Downward sweep from TOA ( $F_0^\downarrow = 0$ ):

$$F_k^\downarrow = \mathcal{T}_k F_{k-1}^\downarrow + \varepsilon_k \sigma_{\text{SB}} T_k^4. \quad (67)$$

### Shortwave (Beer–Lambert)

$$F_{\text{SW}}^\downarrow(k) = \frac{S_0}{4} e^{-\tau_{\text{SW},0} \sigma_k}, \quad F_{\text{SW}}^\uparrow(k) = \alpha_s F_{\text{SW}}^\downarrow(\text{sfc}), \quad (68)$$

with  $\tau_{\text{SW},0} = 0.22$ ,  $S_0 = 1360 \text{ W m}^{-2}$ ,  $\alpha_s = 0.31$ . This follows the Frierson/Isca convention used in the code: only the downward SW beam is absorbed by the atmosphere, while reflected upward SW escapes directly to space.

### Heating rate

Defining  $F_{\text{net}}^\uparrow = F^\uparrow - F^\downarrow$  (upward positive) at each interface, the heating rate for layer  $k$  is

$$\left. \frac{dT}{dt} \right|_{\text{rad},k} = \frac{g}{c_p} \frac{F_{\text{net},k+1/2}^\uparrow - F_{\text{net},k-1/2}^\uparrow}{p_{k+1/2} - p_{k-1/2}}, \quad (69)$$

where  $k + \frac{1}{2}$  is the bottom interface and  $k - \frac{1}{2}$  the top (pressure increasing downward). Positive divergence of upward flux warms the layer.

### RRTMGP Correlated- $k$ Radiation

The RRTMGP backend calls JAX-RRTMGP with correlated- $k$  gaseous optics (128 LW and 112 SW  $g$ -points), a two-stream RTE solver, and optional cloud optics. Default trace-gas concentrations:  $\text{CO}_2=415 \text{ ppmv}$ ,  $\text{CH}_4=1900 \text{ ppbv}$ ,  $\text{N}_2\text{O}=332 \text{ ppbv}$ .

### SBM Convection (Frierson 2007)

#### Moist adiabatic lapse rate

$$\Gamma_m = \frac{R_d T}{c_p p} \frac{1 + L_v q_s / (R_d T)}{1 + L_v^2 q_s / (c_p R_v T^2)}, \quad (70)$$

integrated upward via a trapezoidal predictor–corrector.

#### Saturation mixing ratio (Tetens)

$$e_s = 611.2 \exp\left(\frac{17.67 T_c}{T_c + 243.5}\right), \quad q_s = \frac{\varepsilon e_s}{p - e_s}, \quad \varepsilon = R_d / R_v \approx 0.622. \quad (71)$$

## CAPE and trigger

$$\text{CAPE} = R_d \sum_k \max(0, T_{\text{parcel},k} - T_{\text{env},k}) \frac{\Delta p_k}{p_k}. \quad (72)$$

A smooth sigmoid trigger activates convection:

$$\mathcal{F} = \sigma(\kappa_s(\text{CAPE} - \text{CAPE}_c)), \quad \kappa_s = 0.01 \text{ (J/kg)}^{-1}. \quad (73)$$

## Enthalpy-conserving relaxation

The reference profile  $T_{\text{ref}}$  is obtained by adjusting the moist adiabat so that column-integrated moist enthalpy is conserved:

$$\sum_k [c_p(T_{\text{ref},k} - T_k) + L_v(q_{\text{ref},k} - q_{v,k})] \Delta p_k = 0. \quad (74)$$

Newton iteration (2 steps) solves for the temperature offset. Tendencies:

$$\frac{dT}{dt} = \mathcal{F} \frac{T_{\text{ref}} - T}{\tau_c}, \quad \frac{dq_v}{dt} = \mathcal{F} \frac{q_{\text{ref}} - q_v}{\tau_c}, \quad \tau_c = 7200 \text{ s}. \quad (75)$$

## Precipitation

$$P = -\frac{1}{g} \sum_k \frac{dq_v}{dt} \Delta p_k. \quad (76)$$

## Held–Suarez Forcing

### Equilibrium temperature

$$T_{\text{eq}} = \max\left(200, [315 - \Delta T_y \sin^2 \phi - \Delta \theta_z \ln(p/p_0) \cos^2 \phi] (p/p_0)^\kappa\right), \quad (77)$$

with  $\Delta T_y = 60 \text{ K}$ ,  $\Delta \theta_z = 10 \text{ K}$ .

### Newtonian relaxation

$$\frac{dT}{dt} = -k_T(\sigma, \phi) (T - T_{\text{eq}}), \quad k_T = k_a + (k_s - k_a) \max\left(0, \frac{\sigma - \sigma_b}{1 - \sigma_b}\right) \cos^4 \phi, \quad (78)$$

with  $k_a = 1/(40 \text{ day})$ ,  $k_s = 1/(4 \text{ day})$ ,  $\sigma_b = 0.7$ .

### Rayleigh friction

$$\frac{d\mathbf{v}}{dt} = -k_v(\sigma) \mathbf{v}, \quad k_v = k_f \max\left(0, \frac{\sigma - \sigma_b}{1 - \sigma_b}\right), \quad k_f = 1/(1 \text{ day}). \quad (79)$$

## Louis (1979) Turbulence

### Mixing length

$$l = \frac{\kappa_{\text{vK}} z}{1 + \kappa_{\text{vK}} z/l_{\text{max}}}, \quad \kappa_{\text{vK}} = 0.4, \quad l_{\text{max}} = 100 \text{ m}. \quad (80)$$

### Richardson number

$$\text{Ri} = \frac{N^2}{S^2}, \quad N^2 = \frac{g}{\theta_v} \frac{\partial \theta_v}{\partial z}, \quad S^2 = \left(\frac{\partial u}{\partial z}\right)^2 + \left(\frac{\partial v}{\partial z}\right)^2. \quad (81)$$

### Stability functions

$$f_m(\text{Ri}) = \begin{cases} 1 - \frac{2b \text{ Ri}}{1 + 3bc l^2 |\text{Ri}|^{1/2} / \Delta z^2} & \text{Ri} < 0, \\ \frac{1}{1 + 2b \text{ Ri} / \sqrt{1 + d \text{ Ri}}} & \text{Ri} \geq 0, \end{cases} \quad (82)$$

with  $b = c = d = 5$ . Eddy diffusivities:  $K_m = l^2 S f_m$ ,  $K_h = K_m$ .

### Vertical Diffusion (Implicit Thomas Algorithm)

The backward-Euler system  $-a_k \varphi_{k-1} + b_k \varphi_k - c_k \varphi_{k+1} = d_k$  is solved with the Thomas algorithm.

### Tridiagonal coefficients.

$$a_k = \frac{\Delta t K_{k-1/2}}{\rho_k \Delta z_k \Delta z_{k-1/2}}, \quad c_k = \frac{\Delta t K_{k+1/2}}{\rho_k \Delta z_k \Delta z_{k+1/2}}, \quad b_k = 1 + a_k + c_k. \quad (83)$$

### Forward sweep.

$$w_k = \frac{a_k}{b'_{k-1}}, \quad b'_k = b_k - w_k c_{k-1}, \quad d'_k = d_k + w_k d'_{k-1}. \quad (84)$$

### Back substitution.

$$\varphi_k = \frac{d'_k + c_k \varphi_{k+1}}{b'_k}. \quad (85)$$

### Surface Fluxes

Three bulk flux schemes are available, selected per tile.

### Constant coefficients (default)

$$\tau_x = -\rho C_D |\mathbf{V}| u, \quad \tau_y = -\rho C_D |\mathbf{V}| v, \quad (86)$$

$$\text{SH} = \rho c_p C_H |\mathbf{V}| (T_s - T_a), \quad (87)$$

$$\text{LH} = \rho L_v C_E |\mathbf{V}| (q_s - q_a), \quad (88)$$

with  $C_D = C_H = C_E = 1.5 \times 10^{-3}$  and  $|\mathbf{V}| = \sqrt{u^2 + v^2 + 10^{-4}}$ .

### Monin–Obukhov Similarity Theory (MOST)

Stability-dependent transfer coefficients via iterative Obukhov length computation (Businger–Dyer profiles).

### Obukhov length.

$$L = -\frac{u_*^3 T_v}{\kappa_{\text{vK}} g (\theta_* + 0.61 T_a q_*)}, \quad (89)$$

where  $u_*$ ,  $\theta_*$ ,  $q_*$  are friction velocity, temperature scale, and moisture scale.

**Stability parameter.**  $\zeta = z_{\text{ref}}/L$ .

**Businger–Dyer stability functions.**

$$\psi_m(\zeta) = \begin{cases} 2 \ln\left(\frac{1+x}{2}\right) + \ln\left(\frac{1+x^2}{2}\right) - 2 \arctan x + \frac{\pi}{2}, & \zeta < 0, \quad x = (1 - 16\zeta)^{1/4}, \\ -5\zeta, & \zeta \geq 0, \end{cases} \quad (90)$$

$$\psi_h(\zeta) = \begin{cases} 2 \ln\left(\frac{1+y}{2}\right), & \zeta < 0, \quad y = (1 - 16\zeta)^{1/2}, \\ -5\zeta, & \zeta \geq 0. \end{cases} \quad (91)$$

**Transfer coefficients.**

$$C_D = \left[ \frac{\kappa_{vK}}{\ln(z/z_0) - \psi_m(\zeta)} \right]^2, \quad C_H = \frac{\kappa_{vK}^2}{[\ln(z/z_0) - \psi_m(\zeta)][\ln(z/z_{0t}) - \psi_h(\zeta)]}. \quad (92)$$

**Iterative solver.** The system  $u_*, \theta_*, q_*, z_0, z_{0t}, z_{0q}, L$  is solved iteratively (default 5 iterations) using `jax.lax.fori_loop` for full JAX differentiability.

**COARE 3.0 roughness (Fairall et al. 2003)**

Over ocean, roughness lengths are updated each iteration:

$$z_0 = \alpha_c \frac{u_*^2}{g} + 0.11 \frac{\nu}{u_*}, \quad z_{0t} = \min(1.15 \times 10^{-4}, 5.5 \times 10^{-5} \text{Re}_*^{-0.6}), \quad (93)$$

where  $\alpha_c = 0.011$  is the Charnock parameter and  $\text{Re}_* = u_* z_0 / \nu$ .

**Large & Yeager (2004) roughness**

Empirical neutral drag coefficient as a function of neutral 10-m wind speed  $U_{10N}$ :

$$C_{DN} \times 10^3 = \frac{2.7}{U_{10N}} + 0.142 + 0.0764 U_{10N}, \quad (94)$$

with roughness recovered as  $z_0 = 10 / \exp(\kappa_{vK} / \sqrt{C_{DN}})$ . Stanton and Dalton numbers follow the stability-corrected LY04 parameterisation.

**Additional Turbulence Schemes****Smagorinsky**

Constant eddy diffusivity baseline:  $K_m = K_h = \text{const.}$

**YSU (Yonsei University)**

Nonlocal K-profile boundary-layer scheme with explicit entrainment treatment at the PBL top.

**CLUBB-lite**

Higher-order prognostic moment closure with five prognostic variables ( $\overline{w'^2}$ ,  $\overline{\theta_l'^2}$ ,  $\overline{r_t'^2}$ ,  $\overline{w'\theta_l'}$ ,  $\overline{w'r_t'}$ ) and cloud fraction diagnosis via assumed bivariate Gaussian PDF.

**EDMF Turbulence**

Eddy-diffusivity mass-flux unified PBL framework combining diffusive and convective transport in a single scheme.

## Additional Convection Schemes

### Kuo (1974)

Column moisture-excess scheme partitioning excess moisture between heating (fraction  $\alpha_{\text{heat}}$ ) and moistening ( $1 - \alpha_{\text{heat}}$ ).

### Mass-Flux Convection (Arakawa–Wu Type)

Entraining-detraining plume model with bulk mass flux, CAPE closure, and configurable entrainment/detrainment rates.

### EDMF Convection

Combined eddy-diffusivity and mass-flux framework for shallow and deep convection.

### Deep Convective Adjustment (DCA)

Bottom-to-top adjustment with smooth sigmoid triggers for JAX differentiability.

### Gravity Wave Drag

Six GWD schemes are available:

- **Rayleigh friction:** Simple linear damping above a threshold height.
- **Lindzen (1981):** Orographic GWD with critical-level absorption.
- **McFarlane (1987):** Enhanced orographic scheme with improved saturation.
- **Hines (1997):** Doppler-spread parameterisation for non-orographic GWD.
- **Prognostic Spectral:** Multi-azimuthal, multi-wavenumber spectrum with prognostic wave flux and vertical propagation via `jax.lax.scan`.
- **ML Emulator:** Equinox MLP surrogate for learned GWD.

### Cloud Microphysics

Five microphysics schemes:

- **Kessler (1969):** Warm-rain autoconversion with threshold cloud water.
- **Seifert–Beheng (2001):** Two-moment warm rain with prognostic number concentration.
- **Morrison (2005):** Double-moment ice + liquid scheme.
- **Thompson:** Hybrid-moment scheme with ice crystal habits.
- **Sundqvist (1988) and Xu–Randall (1996)** cloud fraction diagnostics.

`cloud_fraction.compute_cloud_properties` dispatches on `config.scheme` and accepts only "sundqvist" or "xu\_randall"; any other literal raises `ValueError` (no silent fallback). To disable cloud diagnostics, set `cloud_scheme="none"` on the caller's config so that `compute_cloud_properties` is skipped entirely upstream; "none" is not itself a valid scheme inside the dispatch.

## Ocean Dynamics

### Boussinesq Hydrostatic Primitive Equations

In vector-invariant form on the cubed sphere:

$$\frac{\partial u}{\partial t} = (\zeta + f)v - \frac{\partial K}{\partial x} - \frac{1}{\rho_0} \frac{\partial p'}{\partial x} + A_h \nabla^2 u + \frac{\partial}{\partial z} \left( A_v \frac{\partial u}{\partial z} \right) - w \frac{\partial u}{\partial z}, \quad (95)$$

$$\frac{\partial v}{\partial t} = -(\zeta + f)u - \frac{\partial K}{\partial y} - \frac{1}{\rho_0} \frac{\partial p'}{\partial y} + A_h \nabla^2 v + \frac{\partial}{\partial z} \left( A_v \frac{\partial v}{\partial z} \right) - w \frac{\partial v}{\partial z}. \quad (96)$$

The Coriolis force is split: depth-mean  $f\mathbf{v}$  in the barotropic substeps, baroclinic shear  $f\mathbf{v}'$  in the slow tendencies.

### Tracer equations (skew-symmetric)

$$\frac{\partial \phi}{\partial t} = \frac{1}{2} \left[ -\mathbf{v} \cdot \nabla \phi - \nabla \cdot (\phi \mathbf{v}) \right] - w \frac{\partial \phi}{\partial z} + K_h \nabla^2 \phi + \frac{\partial}{\partial z} \left( K_v \frac{\partial \phi}{\partial z} \right). \quad (97)$$

### Free-surface equation

$$\frac{\partial \eta}{\partial t} = - \sum_k \nabla \cdot (h_k \mathbf{v}_k). \quad (98)$$

### Barotropic Solver (Forward–Backward, Semi-Implicit Coriolis)

The barotropic system is solved with split-explicit forward–backward substeps:

$$\frac{\partial \eta}{\partial t} = -\nabla \cdot (H \bar{\mathbf{V}}), \quad \frac{\partial \bar{\mathbf{V}}}{\partial t} = f \hat{\mathbf{k}} \times \bar{\mathbf{V}} - g \nabla \eta + \mathbf{F}_{\text{slow}}. \quad (99)$$

**Semi-implicit Coriolis.** With  $\alpha = \frac{1}{2} f \Delta t_s$ :

$$\bar{U}^{n+1} = \frac{R_u + \alpha R_v}{1 + \alpha^2}, \quad \bar{V}^{n+1} = \frac{R_v - \alpha R_u}{1 + \alpha^2}, \quad (100)$$

where  $R_u = \bar{U}^n + \alpha \bar{V}^n - \Delta t_s g \partial \eta / \partial x$  and  $R_v = \bar{V}^n - \alpha \bar{U}^n - \Delta t_s g \partial \eta / \partial y$ .

### Equation of State (Wright 1997)

The ocean prognostic temperature variable  $T$  is **potential temperature** [degC]. The Wright (1997) EOS computes in-situ density directly from potential temperature  $\theta$ , salinity  $S$  [PSU], and pressure  $p$  [Pa]:

$$\rho = \frac{p + p_0(\theta, S)}{\lambda(\theta, S) + \alpha_0(\theta, S) (p + p_0)}, \quad (101)$$

where

$$\alpha_0 = a_0 + a_1 \theta + a_2 S, \quad (102)$$

$$p_0 = (b_0 + b_4 S) + \theta (b_1 + \theta (b_2 + b_3 \theta) + b_5 S), \quad (103)$$

$$\lambda = (c_0 + c_4 S) + \theta (c_1 + \theta (c_2 + c_3 \theta) + c_5 S). \quad (104)$$

Coefficients follow the MOM6 implementation. The density perturbation is  $\rho' = \rho(\theta, S, p) - \rho_0$ .

### Linear Equation of State

A configurable linear EOS is also available for idealised experiments:

$$\rho = \rho_0 [1 - \alpha_T (T - T_{\text{ref}}) + \beta_S (S - S_{\text{ref}})], \quad (105)$$

with typical values  $\alpha_T = 2 \times 10^{-4} \text{ K}^{-1}$ ,  $\beta_S = 7.4 \times 10^{-4} \text{ PSU}^{-1}$ . The linear EOS is pressure-independent (no hydrostatic pressure coupling iterations required), enabling faster compilation and execution. It is selected via `eos="linear"` in the ocean configuration, with parameters set via a `LinearEOSConfig` NamedTuple. A dispatcher `make_eos_fn()` returns the appropriate EOS callable.

## Hydrostatic Pressure and Vertical Velocity

Baroclinic pressure by downward integration:

$$p'_k = \rho_0 g \eta + \sum_{k'=0}^{k-1} \rho'_{k'} g \Delta z_{k'} + \frac{1}{2} \rho'_k g \Delta z_k. \quad (106)$$

Vertical velocity diagnosed bottom-up from  $w(\text{floor}) = 0$ :

$$w_{k-1/2} = - \sum_{k'=k}^{N-1} \nabla \cdot (h_{k'} \mathbf{v}_{k'}). \quad (107)$$

**Single-level barotropic mode** ( $N_{\text{lev}} = 1$ ). The z-star coordinate supports  $N_{\text{lev}} = 1$ , reducing exactly to the nonlinear shallow-water equations:  $h = H + \eta$ , with no baroclinic modes. Vertical mixing operators return zero for  $N_{\text{lev}} < 2$ . This enables MITgcm-style barotropic gyre experiments with a single depth-integrated layer.

## Ocean Vertical Mixing

Three vertical mixing schemes are available:

### K-Profile Parameterisation (KPP, Large et al. 1994)

Boundary-layer depth  $h_{\text{bl}}$  diagnosed from the bulk Richardson number criterion  $\text{Ri}_b(h) = \text{Ri}_c = 0.3$ . Within the boundary layer, diffusivity follows a cubic shape function:

$$K(z) = h_{\text{bl}} w_* G(\sigma), \quad G(\sigma) = \sigma(1 - \sigma)^2, \quad \sigma = -z/h_{\text{bl}}, \quad (108)$$

where  $w_*$  is a turbulent velocity scale incorporating wind and buoyancy forcing. Non-local transport of temperature and salinity:  $\gamma_T = -C_s (\overline{w'\theta'})_0 / (w_* h_{\text{bl}})$ .

### Richardson-Number Mixing

Interior shear-driven diffusivity:

$$K_v = K_{v,0} (1 + 5 \text{ Ri})^{-1}, \quad \text{Ri} = \frac{N^2}{S^2}. \quad (109)$$

### Constant Mixing

Uniform  $K_v$  (default  $10^{-4} \text{ m}^2 \text{ s}^{-1}$ ) for idealised experiments.

## Ocean Lateral Mixing

### Harmonic (Laplacian) Diffusion

$F_h = A_h \nabla^2 \phi$  with constant or Smagorinsky-type coefficient.

### Biharmonic ( $\nabla^4$ ) Diffusion

$F_h = -A_4 \nabla^4 \phi$ , providing scale-selective damping of grid-scale noise.

### Gent–McWilliams / Redi (GM/Redi)

Mesoscale eddy parameterisation with bolus velocity  $\mathbf{v}^* = -\kappa_{\text{GM}} \nabla_h s$  where  $s$  is the isopycnal slope. Isopycnal diffusion (Redi) rotates the diffusion tensor to align with isopycnal surfaces. Default  $\kappa_{\text{GM}} = 1000 \text{ m}^2 \text{ s}^{-1}$ .

## Ocean Bottom Drag

Two formulations:

- **Linear:**  $\tau_b = -r \mathbf{u}_b$  with configurable  $r$ .
- **Quadratic:**  $\tau_b = -C_d |\mathbf{u}_b| \mathbf{u}_b$  with  $C_d = 2.5 \times 10^{-3}$ .

## Ocean Surface Forcing

Three modes, available on cubed-sphere, lat-lon, and MPAS grids:

- **Prescribed:** Fixed wind stress and heat flux profiles (cosine-latitude, single-gyre, double-gyre, constant).
- **Restoring:** SST/SSS restoring toward climatology (Haney-style) with configurable timescale.
- **Bulk formulas:** COARE-like air-sea fluxes with drag, Stanton, and Dalton coefficients.

On MPAS Voronoi meshes, cell-centred wind stress ( $\tau_x, \tau_y$ ) is projected onto TRiSK edge normals via  $\tau_n = \tau_x \cos \alpha_e + \tau_y \sin \alpha_e$  where  $\alpha_e$  is the edge normal angle. An MPAS-specific physics factory (`make_mpas_ocean_physics`) handles this projection.

## Shortwave Penetration

Subsurface absorption following Jerlov (1968) water type classification (types I, IA, IB, II, III) with two-band exponential decay:

$$I(z) = I_0 [R e^{-z/\zeta_1} + (1 - R) e^{-z/\zeta_2}], \quad (110)$$

where  $R, \zeta_1, \zeta_2$  depend on the water type.

## Ocean Convection

### Enhanced Diffusion

Convective adjustment via enhanced diffusivity  $K_{\text{conv}}$  where  $N^2 < 0$ , with smooth transitions.

### Plume Model

Entraining mass-flux ocean convection with entrainment rate  $\alpha_{\text{plume}} = 0.1$  and background detrainment  $\varepsilon = 10^{-3} \text{ m}^{-1}$ .

## Ocean Conservation Fixers

Grid-specific volume, heat, and salt conservation fixers for cubed-sphere, lat-lon, and MPAS grids. Uniform additive correction per unit volume after each timestep. All three grids compute corrections simultaneously from the original pre-fix state (avoiding order-dependent bias) and upcast to accumulation precision before global reductions to prevent catastrophic cancellation in float32.

**Limitation:** These are uniform additive fixers that violate local conservation and act as spurious globally-uniform diapycnal mixing. The proper solution is flux-form tracer advection with barotropic-baroclinic flux reconciliation (Hallberg 1997, Higdon 2005).

## Tracer Advection Schemes

Tracer transport in `src/legoesm/ocean/advection.py` and `advection_som.py` provides upwind, TVD (Van Leer / Sweby), DST3, PPM, PPM-FCT, WENO5/7, and Prather (1986) Second-Order Moments (SOM) schemes. The volume-weighted variance  $\text{Var}(T)$  on the Eady uniform channel ranks them: `som + Smagorinsky` ( $C_{\text{Smag}}=0.2$ ) preserves 97.1% of  $\text{Var}(T)$  at 600 d in the weak-forcing regime; `tvd` preserves 94% at  $\leq 2\%$  runtime overhead vs. `upwind`

(recommended production default); `ppm_fct` is gated on the sign-split Zalesak/Kuzmin limiter (open issue #9). Forcing regime (strong  $U=0.8\text{ m s}^{-1}$  vs. weak  $U=0.2$ ) dominates absolute mixing; the weak-forcing setup is the scientifically defensible ocean-interior benchmark.

## Realistic-Geometry Spinup

Global Wolfe–Cessi spinups on real ETOPO/GEBCO bathymetry are supported on the lat-lon C-grid via `src/legoesm/ocean/bathymetry.py` (auto-detect ETOPO/GEBCO/ERDDAP/CF, Laplacian smoothing, MEO  $r$ -cap, polar-cap mask, isolated-basin filling, strait enforcement). Stability conditioners: AL81 12-corner-triad PV-flux closure with Le Sommer (2009) partial-cell extension (`pv_flux_al81_partial_cell`); MOM6-style  $\cos^4 \phi$  biharmonic-viscosity scaling (`biharmonic_scaling_factor`, Griffies & Hallberg 2000); SMC03 pressure-gradient force; implicit Crank–Nicolson barotropic solver. Validated 20-yr global-overturning spinup at  $1^\circ$  delivers canonical thermohaline structure (branch `realistic-geometry-full`, PR #224).

## Time Integration

### SSP-RK3 (Shu–Osher 1988)

The three-stage strong-stability-preserving Runge–Kutta method:

$$\mathbf{u}^{(1)} = \mathbf{u}^n + \Delta t F(\mathbf{u}^n), \quad (111)$$

$$\mathbf{u}^{(2)} = \frac{3}{4}\mathbf{u}^n + \frac{1}{4}\mathbf{u}^{(1)} + \frac{1}{4}\Delta t F(\mathbf{u}^{(1)}), \quad (112)$$

$$\mathbf{u}^{n+1} = \frac{1}{3}\mathbf{u}^n + \frac{2}{3}\mathbf{u}^{(2)} + \frac{2}{3}\Delta t F(\mathbf{u}^{(2)}). \quad (113)$$

SSP-RK(4,3) and SSP-RK(5,4) variants are also available.

### Robert-Asselin-Williams (RAW) Time Filter (v3.7)

The leapfrog scheme used by the semi-implicit spectral PE solver requires a time filter to damp the computational mode. The Williams (2009) modification of the Robert–Asselin filter splits the correction between the current and next time levels, restoring second-order accuracy:

$$d^n = \frac{\gamma}{2}(X^{n-1} - 2X^n + X^{n+1}), \quad (114)$$

$$X_{\text{filt}}^n = X^n + (1 - \alpha) d^n, \quad (115)$$

$$X_{\text{filt}}^{n+1} = X^{n+1} + \alpha d^n. \quad (116)$$

With  $\alpha = 0.5$  (default), this is the RAW filter;  $\alpha = 0$  recovers the original Robert–Asselin filter. The original RA filter introduces a first-order phase error that causes slow energy drift over climate-length integrations; the RAW modification eliminates this while preserving the same damping rate for the computational mode.

### Adaptive Hyperdiffusion (v3.7)

The maximum stable coefficient for  $n$ -th order hyperdiffusion under forward Euler is

$$\nu_n = S \cdot \frac{C(n) \Delta x^n}{\Delta t}, \quad C(2) = \frac{1}{2}, \quad C(4) = \frac{1}{8}, \quad C(6) = \frac{1}{48}, \quad (117)$$

where  $S \in (0, 1]$  is a safety factor (default 0.5). The function `adaptive_hyperdiff_coeff(dx_min, dt, order, safety)` eliminates manual tuning of the hyperdiffusion coefficient for each resolution and timestep.

## Split-Explicit Methods

Each outer RK stage computes slow tendencies once, updates  $u, v$  and tracers, then runs  $N_s$  forward-backward acoustic substeps on the fast variables  $(w, \theta', \rho')$  with substep size  $\Delta t_s = \Delta t / N_s$ .

## Land, Ice, and Coupler

### Slab Land Model

#### Energy balance

$$C_s d_s \frac{dT_s}{dt} = (1 - \alpha_l) SW^\downarrow + \varepsilon_l (LW^\downarrow - \sigma_{SB} T_s^4) - SH - LH. \quad (118)$$

#### Bucket hydrology

$$\frac{dW}{dt} = P - E, \quad W \in [0, W_{\max}]. \quad (119)$$

Moisture availability  $\beta = \beta_{\min} + (1 - \beta_{\min}) W / W_{\max}$  modulates evaporation via  $q_{\text{sfc}} = \beta q_s(T_s)$ .

**Surface humidity consistency.** All slab surface tiles (land, sea ice, lake) recompute  $q_{\text{sfc}}$  from the *updated* surface temperature  $T_s^{n+1}$  (and updated  $W^{n+1}$  for land) before constructing the `TileResponse`. This ensures that the returned  $q_{\text{sfc}}$  and  $T_{\text{surface}}$  are thermodynamically consistent at the end of the timestep.

### Sea-Ice Model

#### Surface temperature

$$\rho_i c_i \frac{h_{\text{eff}}}{2} \frac{dT_i}{dt} = Q_{\text{sfc}} - \frac{k_i(T_f - T_i)}{h_{\text{eff}}}, \quad h_{\text{eff}} = \max(h_i, h_{\min}). \quad (120)$$

#### Growth and melt

$$\frac{dh_i}{dt} = \frac{F_{\text{cond}} - F_{\text{ocean}}}{\rho_i L_f}. \quad (121)$$

#### Concentration

$$\frac{dc}{dt} = \begin{cases} (dh/dt)(1 - c)/h_{\text{new}} & \text{growing,} \\ (dh/dt) c / h_{\text{eff}} & \text{melting.} \end{cases} \quad (122)$$

### EVP Rheology (Hunke & Dukowicz 1997)

When `dynamics="evp"`, the ice momentum equation is

$$m \frac{\partial \mathbf{u}_i}{\partial t} = \boldsymbol{\tau}_a + \boldsymbol{\tau}_w - m f \hat{\mathbf{k}} \times \mathbf{u}_i - m g \nabla \eta + \nabla \cdot \boldsymbol{\sigma}, \quad (123)$$

where  $\boldsymbol{\tau}_a, \boldsymbol{\tau}_w$  are air and ocean drag, and  $\boldsymbol{\sigma}$  is the internal stress tensor from an elliptic yield curve with eccentricity  $e = 2$ . Ice strength:

$$P = P^* h \exp(-C(1 - c)), \quad P^* = 27.5 \text{ kPa}, \quad C = 20. \quad (124)$$

The elastic term provides subcycling stability ( $N_{\text{evp}} = 120$  substeps).

## Multi-category ice

For  $N_{\text{cat}} > 1$ , per-category state  $(h_k, T_k, c_k)$  evolves independently under thermodynamics, then Lipscomb (2001) linear remapping redistributes ice across thickness categories.

## Land Carbon Cycle and Stomatal Conductance

### DALEC-990 Carbon Pools

Six prognostic pools: labile  $C_{\text{lab}}$ , foliage  $C_f$ , root  $C_r$ , wood  $C_w$ , litter  $C_{\text{lit}}$ , soil organic matter  $C_{\text{SOM}}$ . GPP via light-use efficiency; decomposition via  $Q_{10}$  temperature sensitivity with moisture limitation.

### Farquhar (1980) Photosynthesis

Assimilation rate:

$$A = \min(W_c, W_j) - R_d, \quad W_c = V_{c,\max} \frac{C_i - \Gamma^*}{C_i + K_c(1 + O/K_o)}, \quad W_j = \frac{J(C_i - \Gamma^*)}{4(C_i + 2\Gamma^*)}, \quad (125)$$

where the minimum is a smooth (differentiable) function.  $V_{c,\max}$  and  $J_{\max}$  follow peaked Arrhenius temperature responses (Bernacchi et al. 2001). Soil moisture stress:  $V_{c,\max} \leftarrow V_{c,\max} \beta_{\text{soil}}$ .

### Stomatal conductance

**Ball–Berry (1987):**  $g_s = g_0 + g_1 \text{ARH}/C_s$ .

**Medlyn (2011):**  $g_s = g_0 + 1.6(1 + g_1/\sqrt{\text{VPD}}) A/C_s$ .

**Jarvis (1976):**  $g_s = g_{s,\max} f(\text{PAR}) f(T) f(\text{VPD}) f(\theta)$  ( $\text{CO}_2$ -independent; used when carbon cycle is inactive).

The coupled  $A$ – $g_s$ – $C_i$  system is solved by 5 fixed-point iterations (unrolled for JAX JIT).

## Plant Functional Types (PFTs)

CLM5-style weighted-average lookup over 17 Plant Functional Types (needleleaf/broadleaf trees, shrubs, grasses, crops), with 12 trainable land parameters per PFT: vegetation albedo, emissivity, roughness length, maximum soil water, soil heat capacity, soil depth, root depth, wilting point, field capacity,  $V_{c,\max,25}$ , LCMA, and stomatal slope  $g_1$ . Each parameter is bounded via sigmoid transforms:

$$p = p_{\min} + (p_{\max} - p_{\min}) \sigma(x_{\text{raw}}). \quad (126)$$

Three parameter provider modes:

- **Constant:** Spatially uniform default values.
- **PFT:** Weighted average  $\bar{p} = \sum_{k=1}^{17} f_k p_k$  from the PFT lookup table.
- **Neural:** 3-hidden-layer FCNN (GELU, 48 hidden units) mapping static features (lat/lon, soil type, elevation, precipitation, temperature, forest age) to land parameters.

## DifferLand Training

End-to-end differentiable land model training with three carbon cycle modes: “none” (disabled), “differland” (prognostic DALEC-990 with trainable allocation and decomposition parameters),

and “seasonal” (prescribed LAI). Phenology uses hemisphere-aware Gaussian windows for leaf emergence/fall. Decomposition follows  $Q_{10}$  temperature sensitivity:

$$k(T, \theta) = k_0 Q_{10}^{(T - T_{\text{ref}})/10} f_w(\theta). \quad (127)$$

NPP allocation is sequential: foliar  $\rightarrow$  labile  $\rightarrow$  root  $\rightarrow$  wood.

## Tile Coupler

Surface fluxes are area-weighted blends over ocean, land, sea-ice, and lake tiles:

$$X_{\text{blend}} = f_{\text{ocn}} X_{\text{ocn}} + f_{\text{land}} X_{\text{land}} + f_{\text{ice}} X_{\text{ice}} + f_{\text{lake}} X_{\text{lake}}. \quad (128)$$

Fluxes are time-averaged over each coupling window before being passed to the atmosphere.

**Ocean albedo.** The constant ocean albedo  $\alpha_{\text{ocn}}$  is controlled by `CouplerConfig.ocean_albedo` (default 0.06). When `ocean_albedo_config.method="zenith"`, the Briegleb (1992) zenith-angle formula is used:

$$\alpha_{\text{ocn}} = \frac{0.026}{1.1 \mu_0^{1.7} + 0.065} + 0.15 (\mu_0 - 0.1)(\mu_0 - 0.5)(\mu_0 - 1.0), \quad (129)$$

where  $\mu_0 = \cos \theta_z$  is the cosine of the solar zenith angle.

**External forcing.** Time-varying greenhouse gases ( $\text{CO}_2$ ,  $\text{CH}_4$ ,  $\text{N}_2\text{O}$ ), total solar irradiance, and monthly zonal-mean ozone and aerosol climatologies can be read from NetCDF files and linearly interpolated in time via the `ExternalForcingConfig` interface. Ozone and aerosol data are loaded but not yet connected to the radiation solver.

**Two-layer lake model.** Epilimnion (mixed layer, default  $h_{\text{epi}} = 5$  m) and hypolimnion (deep layer,  $h_{\text{hypo}} = 20$  m) connected by wind-enhanced vertical mixing:

$$k_{\text{eff}} = k_{\text{mix}} (1 + \alpha_{\text{wind}} |\mathbf{V}|), \quad (130)$$

with surface energy balance driving the epilimnion temperature and mixing flux  $F_{\text{mix}} = \rho c k_{\text{eff}} \Delta T / d_{\text{mid}}$  coupling the two layers. Lake freezing is clamped at  $T_f = 273.15$  K. Bulk flux options: constant coefficients or iterative MOST.

## Machine Learning: Spherical Fourier Neural Operator

The SFNO (ACE/FourCastNet v2 style) operates on the Gaussian grid.

### Architecture

1. **Encoder:** Pointwise linear  $n_{\text{in}} \rightarrow n_{\text{emb}}$ .
2.  **$N$  processor blocks**, each containing:
  - Layer normalisation,
  - SH analysis  $\rightarrow$  learned spectral convolution  $\rightarrow$  SH synthesis,
  - GELU activation,
  - Pointwise MLP (two layers with expansion factor),
  - Skip connection.
3. **Decoder:** Pointwise linear  $n_{\text{emb}} \rightarrow n_{\text{out}}$ .
4. **Big-skip residual:**  $\text{output} = \text{input} + \text{decoder}(\dots)$ .

## Spectral convolution

In the SH coefficient space:

$$\hat{f}_{nm}^{\text{out}} = W_{nm} \hat{f}_{nm}^{\text{in}}, \quad (131)$$

where  $W_{nm} \in \mathbb{R}^{n_{\text{emb}} \times n_{\text{emb}}}$  is a learnable weight matrix per wavenumber pair.

## Neural and Learned Physics

### NeuralPhysics

Equinox MLP column model with hybrid mode blending learned corrections with conventional physics via a configurable blending coefficient.

### LearnedColumn

Per-column physics replacement following Rasp et al. (2018), mapping the full column state to temperature and moisture tendencies via a neural network, trainable end-to-end with the dynamical core.

## Conservation Corrections

### Atmosphere

Dry-air mass is corrected by a uniform adjustment to  $p_s$  that restores the area-weighted global integral:

$$\Delta \bar{p} = \frac{\sum_i A_i p_{s,i}^{\text{new}} - \sum_i A_i p_{s,i}^{\text{old}}}{\sum_i A_i}, \quad p_{s,i}^{\text{corr}} = p_{s,i}^{\text{new}} - \Delta \bar{p}, \quad (132)$$

so that  $\sum_i A_i p_{s,i}^{\text{corr}} = \sum_i A_i p_{s,i}^{\text{old}}$  (global dry mass conserved exactly). Column moisture is rescaled:  $q^{\text{corr}} = q^{\text{new}} \cdot Q_{\text{old}}/Q_{\text{new}}$ .

**Initial-mass anchoring (v3.7).** For long AMIP/CMIP integrations, the fixer can anchor to the *initial* mass  $M_0 = \sum_i A_i p_{s,i}^{\text{init}}$  rather than the previous step's mass, preventing cumulative  $O(\varepsilon)$  drift over millions of timesteps. The correction becomes

$$\Delta \bar{p} = \frac{\sum_i A_i p_{s,i}^{\text{new}} - M_0}{\sum_i A_i}. \quad (133)$$

Set via `model.set_initial_mass(state)` or `anchor_mass_to_initial=True` in the config. All accumulations use float64 regardless of the state precision.

### Ocean

Volume conservation:  $\eta^{\text{corr}} = \eta^{\text{new}} + (\sum \eta_{\text{old}} A - \sum \eta_{\text{new}} A) / \sum A$ .

Heat and salt conservation: uniform additive correction per unit volume.

## Physical Constants

| Symbol               | Name                         | Value                  | Unit                             |
|----------------------|------------------------------|------------------------|----------------------------------|
| $g$                  | Gravitational acceleration   | 9.80616                | $\text{m s}^{-2}$                |
| $R_d$                | Dry-air gas constant         | 287.04                 | $\text{J kg}^{-1} \text{K}^{-1}$ |
| $c_p$                | Dry-air heat capacity        | 1005.0                 | $\text{J kg}^{-1} \text{K}^{-1}$ |
| $R_v$                | Water-vapour gas constant    | 461.5                  | $\text{J kg}^{-1} \text{K}^{-1}$ |
| $L_v$                | Latent heat of vapourisation | $2.501 \times 10^6$    | $\text{J kg}^{-1}$               |
| $\kappa$             | $R_d/c_p$                    | 0.285                  | —                                |
| $p_0$                | Reference pressure           | $10^5$                 | Pa                               |
| $\sigma_{\text{SB}}$ | Stefan–Boltzmann constant    | $5.67 \times 10^{-8}$  | $\text{W m}^{-2} \text{K}^{-4}$  |
| $\kappa_{\text{vK}}$ | von Kármán constant          | 0.4                    | —                                |
| $\Omega$             | Earth rotation rate          | $7.292 \times 10^{-5}$ | $\text{rad s}^{-1}$              |
| $R_{\oplus}$         | Earth radius                 | $6.371 \times 10^6$    | m                                |
| $\rho_0$             | Ocean reference density      | 1025                   | $\text{kg m}^{-3}$               |

## Ocean Biogeochemistry

Two schemes: abiotic (DIC + ALK only) and NPZD ecosystem.

### Carbonate Chemistry and Air-Sea $\text{CO}_2$ Exchange

The ocean surface  $p\text{CO}_2$  is computed from DIC, ALK,  $T$ ,  $S$  via carbonate equilibria. Air-sea flux (Wanninkhof 2014):

$$F_{\text{CO}_2} = k_w K_0 (p\text{CO}_{2,\text{ocean}} - p\text{CO}_{2,\text{atm}}), \quad (134)$$

where  $k_w = 0.251 U_{10}^2 (\text{Sc}/660)^{-1/2}$  is the gas transfer velocity [cm/hr] and  $K_0$  is Henry's law solubility.

### NPZD Ecosystem Model

Four biological tracers:  $\text{NO}_3$ , phytoplankton  $P$ , zooplankton  $Z$ , detritus  $D$  (Fasham et al. 1990).

$$\frac{\partial P}{\partial t} = \mu_P P - G - m_P P, \quad (135)$$

$$\frac{\partial Z}{\partial t} = \gamma_Z G - m_Z Z^2, \quad (136)$$

$$\frac{\partial D}{\partial t} = (1 - \gamma_Z)G + m_P P + m_Z Z^2 - r_D D + w_s \frac{\partial D}{\partial z}, \quad (137)$$

$$\frac{\partial \text{NO}_3}{\partial t} = -\mu_P P + r_D D, \quad (138)$$

where  $\mu_P = \mu_{\text{max}} f(T) \min(L_N, L_I)$  with Michaelis-Menten nutrient limitation  $L_N = \text{NO}_3/(k_N + \text{NO}_3)$  and Beer-Lambert light limitation with chlorophyll self-shading. Grazing  $G$  follows Holling-III. Redfield stoichiometry ( $R_{C:N} = 6.625$ ) couples biology to DIC/ALK.

## CMIP Infrastructure

### CF/CMOR-Compliant Output

The `CFWriter` class produces CF-1.8 NetCDF4 files with CMIP6 Data Reference Syntax (DRS) filenames: `<var>_<table>_<model>_<expt>_<variant>_<grid>.nc`. Four CMOR tables are

provided: **Amon** (33 atmospheric variables including clear-sky radiative fluxes, cloud condensate, and relative humidity), **Lmon** (6 land variables), **Omon** (2 ocean variables: SST and sea-ice fraction), and **Aday** (5 daily atmospheric variables). 3-D fields are interpolated to a standard CMIP6 19-level pressure grid.

## AMIP CMIP6 Forcing Deck

The script `scripts/run_amip_cmip6_deck.py` drives the full CMIP6 AMIP protocol with six prescribed-forcing channels: SST/SIC (HadISST/COBE monthly), annual GHG (`greenhouse_historical_plus` schema, year-indexed CO<sub>2</sub>/CH<sub>4</sub>/N<sub>2</sub>O/CFC-11/CFC-12; consumed by RRTMG only — gray radiation does not respond), ozone (`vmro3` on  $(t, \phi, p)$  either as monthly climatology or multi-decadal CMIP6 transient via the loader’s non-cyclic dispatch), solar TSI plus 14-band fractions (`swflux_14band_cmip6_*` expanded to 112 RRTMG-SW  $g$ -points), aerosol AOD (Kinne zonal monthly), and volcanic AOD (CMIP6 `bc_aeropt_cmip6_volc_*` with 1982/1991 eruptions sampled at correct calendar months). The configs `GHGConfig`, `OzoneConfig`, `SolarConfig`, `AerosolConfig` (`src/legoesm/forcing/external.py`) all carry `start_year` so simulation days map onto file absolute time axes via `_simday_to_file_day`.

The deck physics defaults are RRTMG + Sundqvist cloud fraction + Sundqvist large-scale condensation + SBM + Louis (Kessler microphysics is currently disabled in this path because it produces NaN winds at day ~2). When forcing files are absent the user must pass `--auto-generate` to invoke `scripts/generate_amip_forcing.py` for synthetic CMIP6-shape replacements. `scripts/smoke_test` exercises the dispatch path on cubed-sphere (centred / FV3 / C-D), lat-lon (centred / FV3 / C-grid), Gaussian spectral, and Voronoi/MPAS using gray radiation with aerosol/volcanic disabled; it does not validate the full forcing stack on every grid. Voronoi/MPAS additionally requires `--turbulence none` (TKE/MPAS edge-wind mismatch) and `--dt 60` (hidden hydrostatic-dycore CFL). Spectral and MPAS paths bypass `DiagnosticCollector`.

## Experiment Templates

Six CMIP6-style experiments with built-in GHG time series:

| Template                | Period         | Forcing        | CO <sub>2</sub> range |
|-------------------------|----------------|----------------|-----------------------|
| <code>piControl</code>  | 1850– $\infty$ | Fixed 1850     | 284.3 ppmv            |
| <code>historical</code> | 1850–2014      | Transient      | 284→397 ppmv          |
| <code>ssp245</code>     | 2015–2100      | SSP2-4.5       | 397→600 ppmv          |
| <code>ssp585</code>     | 2015–2100      | SSP5-8.5       | 397→1135 ppmv         |
| <code>amip</code>       | 1979–2014      | Prescribed SST | Transient             |
| <code>1pctCO2</code>    | 1–150          | 1%/yr increase | 284→1277 ppmv         |

## Restart and Reproducibility

Each restart file is accompanied by a `.meta.json` sidecar containing: SHA-256 state digest (sorted, concatenated array bytes), SHA-256 config hash, platform identifier, JAX version, x64 mode, and git hash. The function `verify_reproducibility()` compares two restart files and reports any discrepancies.

## Tuning Guide

A registry of 16 tunable parameters across five categories (dynamics, radiation, convection, diffusion, surface) with valid ranges, default values, sensitivity ratings, and physical notes. `validate_tuning(config)` checks CFL consistency, range violations, and conflicting options. `recommended_params(resolution, nlev)` provides resolution-appropriate defaults.

## OMIP Simulation Pipeline

The `run_omip.py` script provides a complete Ocean Model Intercomparison Project (OMIP) simulation pipeline supporting cubed-sphere, lat-lon, MPAS, and spectral grids. Features include WOA18-based initialisation with fallback, Haney-style SST/SSS restoring toward climatology, configurable physics presets (“none”, “minimal”, “full” with KPP, GM/Redi, bottom drag, shortwave penetration), per-grid resolution and timestep defaults, and diagnostic extraction with wall-time tracking.

## 4D-Var Data Assimilation (v3.9)

The model includes a fully differentiable 4D-Var data assimilation system that exploits JAX’s automatic differentiation to compute exact gradients of the cost function without hand-coded adjoints.

### Cost Function

The 4D-Var cost function minimised over the assimilation window  $[t_0, t_N]$  is

$$J(\mathbf{x}_0) = \frac{1}{2}(\mathbf{x}_0 - \mathbf{x}_b)^T \mathbf{B}^{-1}(\mathbf{x}_0 - \mathbf{x}_b) + \frac{1}{2} \sum_{i=1}^N (\mathcal{H}_i[\mathcal{M}_{0 \rightarrow i}(\mathbf{x}_0)] - \mathbf{y}_i)^T \mathbf{R}_i^{-1}(\mathcal{H}_i[\mathcal{M}_{0 \rightarrow i}(\mathbf{x}_0)] - \mathbf{y}_i), \quad (139)$$

where  $\mathbf{x}_b$  is the background state,  $\mathbf{B}$  the background error covariance,  $\mathcal{M}_{0 \rightarrow i}$  the nonlinear model propagator,  $\mathcal{H}_i$  the observation operator at time  $t_i$ ,  $\mathbf{y}_i$  the observations, and  $\mathbf{R}_i$  the observation error covariance. Gradients  $\nabla_{\mathbf{x}_0} J$  are computed via `jax.grad` through the full forward model trajectory.

### Control Vector

The control vector comprises perturbations to:

- Horizontal wind components ( $u$ ,  $v$ ),
- Temperature ( $T$ ),
- Specific humidity ( $q_v$ ),
- Surface pressure ( $p_s$ ).

Increments are applied in model space and can optionally be preconditioned by the square root of  $\mathbf{B}$  (i.e.  $\mathbf{x}_0 = \mathbf{x}_b + \mathbf{B}^{1/2} \boldsymbol{\chi}$  with minimisation in  $\boldsymbol{\chi}$ -space).

### Background Error Covariance

Four background error covariance formulations are implemented:

1. **Diagonal:**  $\mathbf{B} = \text{diag}(\sigma_b^2)$ , the simplest variance-only model.
2. **Diffusion:**  $\mathbf{B} = \boldsymbol{\Sigma} \mathbf{C} \boldsymbol{\Sigma}$  where  $\mathbf{C}$  is a correlation matrix defined implicitly via iterated Laplacian diffusion on the model grid.
3. **Spectral:**  $\mathbf{B}$  is diagonal in spectral space, with variance specified per total wavenumber; correlations arise naturally from the spectral truncation.
4. **Hybrid:**  $\mathbf{B} = \beta_1 \mathbf{B}_{\text{static}} + \beta_2 \mathbf{B}_{\text{ensemble}}$ , blending a static covariance with flow-dependent ensemble covariance.

### Observation Operators

Three observation operator types are provided:

- **Direct:** identity mapping from model grid points to observed quantities.

- **Interpolating:** bilinear (horizontal) and linear (vertical) interpolation from the model grid to observation locations.
- **Composite:** chains multiple operators (e.g. interpolation followed by a forward radiative transfer calculation).

## Minimiser

The cost function is minimised using preconditioned gradient descent. The preconditioner is the square root of  $\mathbf{B}$ , transforming the problem to  $\chi$ -space where the background term becomes isotropic. The minimiser supports configurable iteration counts and convergence tolerances, and all operations are JIT-compiled.

## DA Cycling and Incremental Analysis

For operational-style cycling, the analysis from each window serves as the background for the next. An incremental analysis update (IAU) option distributes the analysis increment gradually over the subsequent forecast window rather than inserting it as an instantaneous shock, reducing initialisation imbalance and gravity-wave noise.

## References

- Audusse, E., F. Bouchut, M.-O. Bristeau, R. Klein, and B. Perthame (2004)  
A fast and stable well-balanced scheme with hydrostatic reconstruction for shallow water flows. *SIAM J. Sci. Comput.*, **25**, 2050–2065.
- Ball, J.T., I.E. Woodrow, and J.A. Berry (1987)  
A model predicting stomatal conductance and its contribution to the control of photosynthesis under different environmental conditions. *Prog. in Photosynthesis Res.*, **4**, 221–224.
- Bernacchi, C.J., E.L. Singsaas, C. Pimentel, A.R. Portis, and S.P. Long (2001)  
Improved temperature response functions for models of Rubisco-limited photosynthesis. *Plant Cell Environ.*, **24**, 253–259.
- Bourke, W. (1972)  
An efficient, one-level, primitive-equation spectral model. *Mon. Wea. Rev.*, **100**, 683–689.
- Businger, J.A., J.C. Wyngaard, Y. Izumi, and E.F. Bradley (1971)  
Flux-profile relationships in the atmospheric surface layer. *J. Atmos. Sci.*, **28**, 181–189.
- Colella, P. and P.R. Woodward (1984)  
The Piecewise Parabolic Method (PPM) for gas-dynamical simulations. *J. Comput. Phys.*, **54**, 174–201.
- Farquhar, G.D., S. von Caemmerer, and J.A. Berry (1980)  
A biochemical model of photosynthetic CO<sub>2</sub> assimilation in leaves of C3 species. *Planta*, **149**, 78–90.
- Fasham, M.J.R., H.W. Ducklow, and S.M. McKelvie (1990)  
A nitrogen-based model of plankton dynamics in the oceanic mixed layer. *J. Mar. Res.*, **48**, 591–639.
- Fairall, C.W., E.F. Bradley, J.E. Hare, A.A. Grachev, and J.B. Edson (2003)  
Bulk parameterization of air–sea fluxes: Updates and verification for the COARE algorithm. *J. Climate*, **16**, 571–591.

- Frierson, D.M.W., I.M. Held, and P. Zurita-Gotor (2006)  
A gray-radiation aquaplanet moist GCM. Part I: Static stability and eddy scale. *J. Atmos. Sci.*, **63**, 2548–2566.
- Frierson, D.M.W. (2007)  
The dynamics of idealized convection schemes and their effect on the zonally averaged tropical circulation. *J. Atmos. Sci.*, **64**, 1959–1976.
- Jiang, G.-S. and C.-W. Shu (1996)  
Efficient implementation of weighted ENO schemes. *J. Comput. Phys.*, **126**, 202–228.
- Hunke, E.C. and J.K. Dukowicz (1997)  
An elastic-viscous-plastic model for sea ice dynamics. *J. Phys. Oceanogr.*, **27**, 1849–1867.
- Held, I.M. and M.J. Suarez (1994)  
A proposal for the intercomparison of the dynamical cores of atmospheric general circulation models. *Bull. Amer. Meteor. Soc.*, **75**, 1825–1830.
- Large, W.G. and S.G. Yeager (2004)  
Diurnal to decadal global forcing for ocean and sea-ice models: The data sets and flux climatologies. *NCAR Technical Note*, NCAR/TN-460+STR.
- Lin, S.-J. (2004)  
A “vertically Lagrangian” finite-volume dynamical core for global models. *Mon. Wea. Rev.*, **132**, 2293–2307.
- Lipscomb, W.H. (2001)  
Remapping the thickness distribution in sea ice models. *J. Geophys. Res.*, **106**(C7), 13989–14000.
- Louis, J.-F. (1979)  
A parametric model of vertical eddy fluxes in the atmosphere. *Bound.-Layer Meteor.*, **17**, 187–202.
- Medlyn, B.E. et al. (2011)  
Reconciling the optimal and empirical approaches to modelling stomatal conductance. *Global Change Biol.*, **17**, 2134–2144.
- Mouallem, J., L.M. Harris, and R. Benson (2023)  
A duo-grid methodology for cubed-sphere shallow water models. *J. Comput. Phys.*, **485**, 112097.
- Putman, W.M. and S.-J. Lin (2007)  
Finite-volume transport on various cubed-sphere grids. *J. Comput. Phys.*, **227**, 55–78.
- Shu, C.-W. and S. Osher (1988)  
Efficient implementation of essentially non-oscillatory shock-capturing schemes. *J. Comput. Phys.*, **77**, 439–471.
- Simmons, A.J. and D.M. Burridge (1981)  
An energy and angular-momentum conserving vertical finite-difference scheme and hybrid vertical coordinates. *Mon. Wea. Rev.*, **109**, 758–766.
- Skamarock, W.C. and J.B. Klemp (2008)  
A time-split nonhydrostatic atmospheric model for weather research and forecasting applications. *J. Comput. Phys.*, **227**, 3465–3485.

- van Leer, B. (1979)  
Towards the ultimate conservative difference scheme. V. A second-order sequel to Godunov's method. *J. Comput. Phys.*, **32**, 101–136.
- Wanninkhof, R. (2014)  
Relationship between wind speed and gas exchange over the ocean revisited. *Limnol. Oceanogr. Methods*, **12**, 351–362.
- Wright, D.G. (1997)  
An equation of state for use in ocean models: Eckart's formula revisited. *J. Atmos. Ocean. Tech.*, **14**, 735–740.